

Article

Per-Field Categorical G-Code Recoverability from Multi-Modal Sensor Embeddings on a Desktop CNC Mill: A Single-Platform Characterization Study

Stephen S. Eacuella^{1,*} , Romesh S. Prasad¹ and Manbir S. Sodhi¹

¹ Department of Industrial and Systems Engineering, University of Rhode Island, Kingston, RI 02881, USA

* Correspondence: seacuella@uri.edu

Version August 28, 2026 submitted to Sensors

Abstract: Controller-resident tampering can alter executed G-code while the loaded file's hash stays valid. We characterize **per-field categorical recoverability** of G-code from a desktop CNC mill's 4 Hz seven-modality sensor stream, using a grammar-constrained six-head Transformer decoder over a frozen denoising encoder under five-fold file-level cross-validation. Categorical fields are recoverable under reference-conditioned (teacher-forced) evaluation, the command head at ~ 0.89 accuracy; measured free-running categorical accuracy collapses to 0.48. The decoder adds +22 pp over the history-free operation-class modal (0.795 vs. 0.572, command-token positions) and $\approx +13$ pp over a history-matched Markov null (0.662); the increment is a 4-of-5-fold effect (at or below the class prior on one fold) and an *upper* bound on sensor-driven recovery. By contrast, generative reconstruction collapses: token accuracy falls $0.78 \rightarrow 0.21$ and digit $0.59 \rightarrow 0.10$ under autoregression, with whole-sequence exact-match at 0.026 under exposure-bias mode collapse, on a corpus in which 77.7% of blocks occupy ≤ 1 sample period. Under open-vocabulary leave-one-class-out evaluation, command recovery collapses to 0.213. Generative coordinate reconstruction is unsupported (source-entropy and sub-Nyquist bounds; companion paper). Tamper detection is deferred to a companion security study; the posture is forensic-support post-hoc audit in a single-platform, closed-world scope, not real-time intrusion detection.

Keywords: CNC machining; G-code reconstruction; multi-modal sensor fusion; manufacturing cybersecurity; tamper detection; grammar-constrained inference; forensic audit

1. Introduction

A Computer Numerical Control (CNC) milling machine drives a cutting tool along G-code-specified multi-axis toolpaths to micrometer tolerances. Every cut leaves a multi-modal sensor signature in machine-frame vibration, acoustic emission, motor-current draw, and ambient temperature and illumination. Whether that signature is informative enough to *recover* the instruction stream line by line, without controller access, is the question this paper takes up, with consequences for sensor-side recoverability bounds and deployment-grade audit of contested runs.

We study a single desktop CNC milling platform (Bantam Tools Explorer mill; air-cut runs and dry UHMW-PE active-cut runs; 4 Hz envelope-extracted sensor stream); cross-platform replication on a production-class VMC is future work (Section 8.4) and a precondition for any deployment claim beyond this scope. The intended application is **forensic-support post-hoc audit**: a controller-independent record of executed instructions, reviewed offline against the controller log when a run is flagged, complementing file-hash signing and controller attestation, not a real-time intrusion-detection system. It also supports offline process verification and human-in-the-loop triage in networked manufacturing

environments [1–6], over a narrow subset of G-code substitutions (command, axis, and motion-sign edits) within the trained toolpath envelope.

Three boundaries are out of scope. First, real-time autonomous SOC tier-1 alerting is *not* a supported deployment posture on this stack (Section 7.4). Second, fine-grained numeric-coordinate and feed-rate substitutions are not value-recoverable on the present 4-Hz sensor stack by an information-theoretic bound (Section 7.4); whether gross parameter deviations remain detectable as out-of-envelope anomalies is assessed in the companion security study. Third, the audit posture is bound to *in-distribution* inputs, closed-world over the V8 token vocabulary (Section 3.2) and the nine training-time operation classes (Section 5.1); novel-command and novel-operation-class inputs fall outside this envelope (LOCO collapse, Section 6.8). The wider cost envelope a richer sensor stack could catch [7,8] frames future cross-platform replication, not the present system.

Sensor monitoring of CNC machines is mature but coarse-grained (Section 2.1). Whether the specific G-code instruction executing at this moment is identifiable from sensors alone has been largely absent from the CNC literature, despite implications for tamper detection, for the dual-use side-channel leakage of manufacturing instructions [9–14], and for root-cause analysis on contested runs.

Why CNC milling is harder than the additive-manufacturing analogue.

In the closest analog, side-channel toolpath reconstruction in additive manufacturing [9–11], Fused Deposition Modeling deposits material one axis at a time and sensors can be placed freely. CNC milling is a harder signal environment: simultaneous multi-axis interpolation, nonlinear force-engagement, structural vibration through the machine frame, and high-energy chip ejection corrupt sensor signals and constrain sensor placement [7,15]. The mapping is also many-to-one, since many instructions produce similar signatures when cutting parameters coincide and one instruction can be cut at different speeds, depths, or materials.

Building on prior work in our group.

The sensor encoder used here is a prior contribution achieving 96.3% accuracy on nine-class CNC operation classification via a multi-modal denoising autoencoder over six distributed sensor boards [16], establishing representational competence at *operation-class* granularity. A companion contribution defined a hybrid grammar-based tokenization of G-code for sequence models [17]. This paper takes the encoder as a fixed feature extractor and the grammar as a vocabulary, asking what its per-window latent carries about the row-level instructions that drove the cut.

Three challenges specific to this task.

Direct G-code generation from sensor signals raises three issues that off-the-shelf sequence-to-sequence models [18–21] do not address. (i) A 64-second window straddles tens to hundreds of G-code rows, so a naive window-level target destroys row-level information. (ii) The numeric vocabulary contains thousands of bucket tokens, most absent from any one training fold. (iii) Positional metadata (window index, file identifier) is a shortcut that inflates accuracy without proving sensor-driven recovery [22,23].

Research questions.

RQ1. Per-field recoverability (window-aggregated). Which components of a G-code line (command identity, axis presence, motion direction, numeric value) are recoverable from sensor data alone, and with what fidelity? At the released 4-Hz rate, 77.7% of G-code rows occupy ≤ 1 sample period (Section 8.2), so any answer is a *window-aggregated row-modal recovery* claim, not a sub-sample-resolved per-row claim.

RQ2. The metadata-versus-sensor split. Does exposing positional metadata (window index, file identifier) to the decoder improve recovery, and which fields benefit?

RQ3. Sensor-modality contribution. Which of seven sensor groups (accelerometer, gyroscope, magnetometer, environmental, color, audio-RMS, electrical) carry the strongest instruction-level signal?

RQ4. Cross-class generalization. Does the decoder generalize to machining operation classes it never saw during training (leave-one-class-out)?

Headline findings (stated as bounds, not raw accuracy).

The central quantities are not the closed-vocabulary ~ 0.89 command accuracy but two bounds that frame it. Above the operation-class modal predictor, the decoder adds a +22 pp class-prior-controlled within-class command increment (Section 7.1.0.2), which we report as an *upper* bound on sensor-driven recovery; it is unchanged under a test-derived class-modal null but is a 4-of-5-fold effect, at or below its class prior on one fold, and reduces to $\approx +13$ pp against a history-matched sensor-free Markov null (Section 7.1.0.2). On an open-vocabulary leave-one-class-out holdout, command recovery collapses to a floor of 0.213 (Section 6.8). End-to-end generative reconstruction is at floor (autoregressive whole-sequence exact-match 0.026). We therefore reserve “recovery” for the per-field categorical heads and treat generative coordinate reconstruction as unsupported on this stack.

Contributions.

(1) Per-row supervised target formulation. Each sensor window is mapped to the single G-code line firing within it, yielding $\sim 19,500$ training samples per fold versus ~ 300 for a window-level alternative and removing within-window label ambiguity; the headline configuration trains on the complementary full-window variant (Section 6.6.3).

(2) G-Code Decoder architecture. A Transformer decoder with six structured prediction heads (token, type, motion-command, parameter-type, sign, digit) over the frozen latent of a prior multi-modal denoising encoder [16], grammar-constrained at the token head so that structurally invalid transitions are masked at runtime [24–26].

(3) Per-axis recoverability characterization reporting for each field whether the axis is detected, the motion direction is correct, and the numeric value is recovered. The corpus supports X, Y, Z, and R; F at thin support; S, I, and J are not evaluable (Table 3).

(4) Ablation matrix over positional-metadata access, per-row vs. full-window targets, sequence-classifier prior, vocabulary precision, each of seven sensor modalities, and a no-numeric retraining (Section 6.6.2) that decomposes the autoregressive mode-collapse into a numeric-desynchronization component and a dominant intrinsic structural-stream collapse.

(5) Cross-class generalization study: leave-one-class-out across nine operation classes, testing whether the decoder memorizes per-class shortcuts or generalizes within the present envelope.

(6) From-scratch Transformer seq2seq baseline (Section 6.9) scoping the architectural contribution to autoregressive deployment-mode inference, where the frozen encoder plus grammar-constrained decoder plus structured heads buy the lift. A companion limit paper [27] accounts for *why* the recoverability pattern takes this shape (Section 2.3).

Scope statement.

Every quantitative result is conditional on this single desktop platform, material class, and 4 Hz sensor stream; cross-platform replication is future work (Section 8.4).

2. Related Work

2.1. CNC Process Monitoring and Multi-Modal Sensor Fusion

Sensor-based machining monitoring is mature. Force, vibration, acoustic emission, temperature, and motor current are complementary modalities for tool-condition assessment [7], historically via hand-crafted Fourier and wavelet features [28,29] and later via end-to-end deep feature learning [15,30]. Convolutional networks over vibration spectrograms and raw multi-channel signals address bearing fault detection [31], surface-roughness classification [32,33], and CNC anomaly detection [34]. Multi-modal fusion spans early, late, and intermediate strategies [35,36], with cross-modal attention useful when modalities are complementary rather than redundant [37,38]; motor-current and servo-drive signatures are an established adjacent modality for load and toolpath inference [7,15], one of the seven modality groups fused by the encoder used here.

Digital-twin and digital-shadow verification compares observed machine state against a simulated reference of the *intended* program [39]; the present work instead reconstructs the executed stream from sensors alone, without a reference model or trusted controller, on orthogonal trust assumptions. Two gaps persist relative to instruction-level recovery. Prior target labels are session-level (tool worn / not worn) or coarse class labels (turning vs. milling) [40], with per-row instruction targets largely absent. Sensor-group ablations that quantify marginal contribution [22,23] are also uncommon in CNC monitoring, leaving open which sensors carry instruction-level as distinct from operation-class signal.

2.2. Sequence-to-Sequence Modeling and Grammar-Constrained Decoding

Sequence-to-sequence models, established for natural-language translation [18–20], have been applied to speech [21], time series [41], and robotics [42]. Long Short-Term Memory networks [43] remain common for sequential industrial data with streaming, causally ordered inference [44], and denoising autoencoders [45] learn representations robust to mechanical, electromagnetic, and thermal sensor noise.

When the output follows a known grammar, constrained decoding prunes structurally invalid tokens at inference time, from parser-guided text-to-SQL [24] and general code generation [46,47] to near-zero-overhead per-token masking via reusable automata [25,26]. Geng *et al.* [48] recover structured outputs from off-the-shelf pretrained models without task-specific fine-tuning. G-code is amenable to the same treatment, since instructions are an optional command token followed by address-value pairs under first-order ordering constraints [17,49]. Recent large-language-model G-code generation for additive manufacturing [50–52] works from natural-language descriptions or images, a distinct task that ignores sensor-conditioned recovery.

2.3. Side-Channel Reconstruction of Manufacturing Instructions

The closest analog is side-channel reconstruction of toolpaths in additive manufacturing. Al Faruque *et al.* [9] recovered Fused Deposition Modeling print geometry from acoustic emissions, Song *et al.* [10] reproduced printed objects from a smartphone microphone alone, and Chhetri *et al.* [11] identified kinetic anomalies indicative of attacks. Later work reconstructs full part geometries from 3D printers via optical [14] and combined acoustic-magnetic [13] side-channels; Sturm *et al.* [3] and Yampolskiy *et al.* [4] survey cyber-physical attack taxonomies.

That literature typically uses *raw acoustic emission* at kHz–MHz rates, whereas the present work uses an *audio-RMS envelope* scalar feature at 4 Hz that retains cutting-sound energy magnitude but discards the spectral content driving most prior AM methods, making the recovery reported here not directly comparable and more conservative in its sensor demands. Because CNC milling is a harder sensor environment than additive manufacturing (Section 1), those methods do not transfer directly. Kimmell *et al.* [6] detect control-injection attacks via energy-anomaly thresholds, but perform

binary detection rather than instruction-level reconstruction. To our knowledge, no prior work reports per-row G-code recovery from CNC sensor data with structured-output evaluation.

A companion limit paper [27] develops the information-theoretic account of *why* per-field recoverability stratifies as it does, via the per-head and per-toolpath source entropy of the CNC instruction stream and its 4 Hz sub-Nyquist floor; the present work treats those bounds as given.

2.4. Manufacturing Cybersecurity and Process Verification

Adversarial G-code substitution in PLC and CNC controllers is an established attack surface [6], which motivates a controller-independent, out-of-band record of the executed instruction stream as an integrity-checking baseline that controller-resident malware cannot subvert, subject to the sensor path itself being trustworthy (Section 7.4). The present work characterizes only which components of that stream are reconstructable on present-day sensor hardware; the tamper-detection apparatus built on the record, its guarantees, and its sensor-integrity assumptions are deferred to a companion security paper.

Positioning.

Against representative prior work, AM side-channel studies [9–11,14] report toolpath-geometry or binary-anomaly targets without field-level or leave-one-class-out evaluation; CNC studies [6,33,34] and the encoder contribution [16] report class-level or binary targets; file-hash signing [2,53] and controller attestation [54] are orthogonal non-sensor defenses. This work is the only entry with a per-row token-sequence target, field-level evaluation, and a nine-class LOCO test under 5-fold file-level cross-validation. To our knowledge this is the first per-instruction, per-field categorical recovery of CNC *milling* G-code from a multi-modal sensor stream; the cross-validation and LOCO protocol support that claim rather than standing as a co-equal novelty leg.

3. Problem Formulation

3.1. Task Definition

Let $W \in \mathbb{R}^{T \times C}$ denote a sensor window of T samples at C channels, aligned to a CNC milling operation, and g the G-code line executing within W . The task is to learn a mapping

$$f_{\theta} : W \mapsto \hat{g}, \quad \hat{g} = (\hat{t}_1, \hat{t}_2, \dots, \hat{t}_L),$$

where $\hat{g}$ is a sequence of L G-code tokens from a vocabulary $\mathcal{V}$. Training minimizes a multi-head cross-entropy loss with the auxiliary structured heads of Section 4.3; each target ends in an end-of-sequence token. At inference, the decoder generates tokens autoregressively under a grammar mask that zeros structurally invalid transitions in the legacy-token head.

3.2. Vocabulary and Token Types

Following CNC-shop convention, each G-code line is a *block*; motion blocks are *rapid moves* (G0, traverse without cutting) or *cutting moves* (G1/G2/G3, linear and arc feeds). Both are tokenized identically by a hybrid scheme disentangling structural primitives from numeric values [17,49]. The vocabulary has four token classes:

- **SPECIAL tokens:** PAD, BOS, EOS, UNK, MASK.
- **COMMAND tokens:** literal motion or machine commands (G0, G1, G2, G3, M3, M5, M30, ...).
- **PARAM tokens:** address letters introducing an axis or rate parameter (X, Y, Z, I, J, K, R, F, S, P).
- **NUMERIC tokens:** bucketed numeric values, one bucket per (axis, magnitude) pair. The vocabulary contains 1,048 NUM_X buckets, 1,086 NUM_Y buckets, and 231 NUM_Z buckets, with smaller sets for the remaining axes; total vocabulary size 2,418. Per-axis quantization precisions, the bucket-index formula, and the out-of-range fallback are in Appendix A.

For example, `G1 X1.4919 Y1.4848 F22.` becomes `BOS G1 X NUM_X_1492 Y NUM_Y_1485 F NUM_F_22 EOS`. Bucketization discretizes to a fixed precision; the per-position digit head (Section 4.3) supplies an auxiliary regression-style signal mitigating the discretization loss.

3.3. Per-Row Target Formulation

A naive target assigns one G-code label per window. Here a 256-sample window at 4 Hz spans 64 seconds and averages 60 G-code rows (median 29, max 237), so a window-level target aggregates across rows and discards row-level distinctions such as an X positive versus negative move. We instead emit one (W, g_r) pair per row r firing within W , with g_r that row's token sequence. This per-row formulation removes within-window label ambiguity (each pair has exactly one correct target), raises the training-sample count by an order of magnitude ($\sim 19,500$ per fold versus ~ 300 window-level), and decouples it from the windowing schedule, blocking positional-shortcut label inference on `(window_index, source_file)`.

A complementary *full-window* formulation (Section 6.6.3) keeps one sample per window with a multi-line target of all fired rows in temporal order: fewer samples but a richer per-sample signal, testing whether row-level autoregressive context closes any per-row recovery gap. The headline configuration (Table 1) trains on this full-window variant, which restores within-window row identifiability; per-row remains the preferred autoregressive deployment mode (Section 7.2.0.2).

3.4. Evaluation Metrics

We report seven metrics:

1. **Token accuracy:** proportion of non-padding positions whose argmax matches the target; the headline metric.
2. **Sequence accuracy:** proportion of test samples with *every* non-padding token correct; a strict exact-match metric.
3. **Type accuracy:** per-token accuracy on the 4-way SPECIAL/COMMAND/PARAM/NUMERIC type head.
4. **Command accuracy:** per-token accuracy on the 6-way motion-command head at COMMAND positions.
5. **Parameter-type accuracy:** per-token accuracy on the 10-way axis-letter head at PARAM positions.
6. **Sign accuracy:** per-position accuracy on the 3-way (positive, negative, absent) motion-sign head.
7. **Numeric (digit) accuracy:** per-digit-position correctness pooled across all six digit positions in NUM tokens; the companion regression measure is the reconstructed-coordinate mean absolute error (MAE).

Macro precision, recall, and F1 are computed per head with scikit-learn [55] on the per-token confusion matrix; the headline table reports precision and F1.

Teacher-forced vs. autoregressive evaluation.

Teacher-forced (TF) evaluation supplies the ground-truth previous token at every position (the training objective and sweep-time metric); *autoregressive* (AR) evaluation consumes the model's own predictions, measuring end-to-end generation when no controller log anchors the sequence. The categorical heads (type, command, parameter-type, sign) are evaluated from the ground-truth-conditioned forward pass in both regimes, so the headline table reports a single number per categorical head; because these heads read the decoder hidden state, which is computed from the conditioning token stream (Section 4.3.1), their accuracy under free-running generated-stream conditioning is a distinct, measured quantity: command falls to 0.483 ± 0.049 , type to 0.551 ± 0.087 , and parameter-type to 0.533 ± 0.046 (`audit/ar_categorical_freerunning.json`). The autoregressive-token heads score lower under AR because exposure-bias errors compound along the sequence; the headline table reports both regimes for them.

3.5. Threat Model (Companion Study)

The application motivating per-field recoverability is out-of-band verification of the executed instruction stream against controller-resident tampering. The full threat model — trusted and untrusted elements, the attack-class taxonomy, and per-class detectability — is developed in a companion security study (in preparation); the present paper characterizes only which G-code fields are recoverable, the precondition for any such monitor.

4. Method

4.1. System Overview

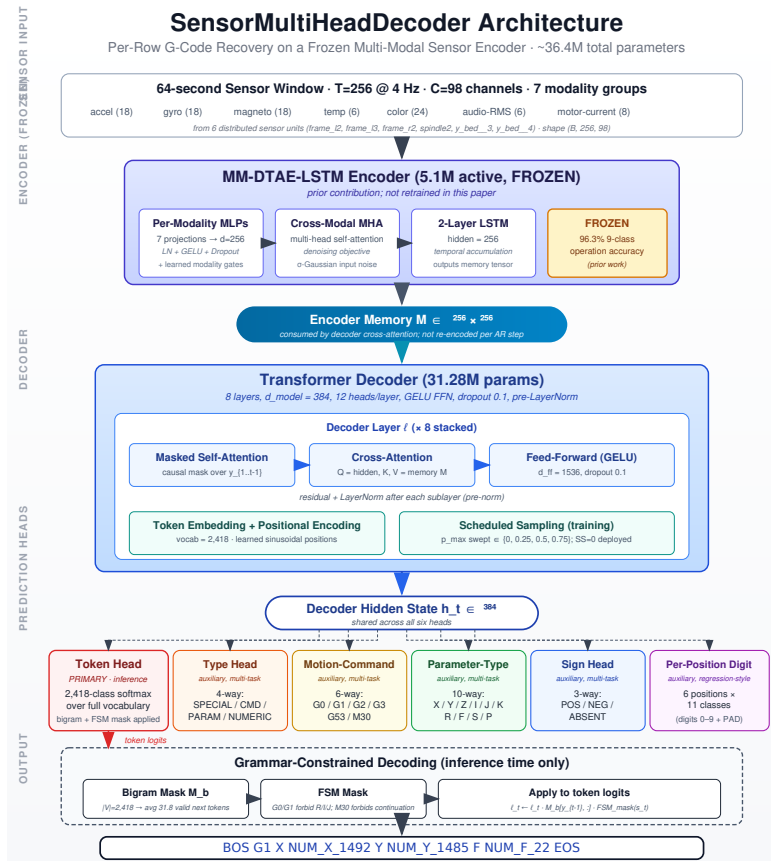


Figure 1. System overview. A 64-second sensor window (256 samples $\times$ 98 channels across seven modality groups) is mapped by the frozen MM-DTAE-LSTM encoder (≈ 5.1 M active parameters) to a 256×256 memory tensor. An 8-layer Transformer decoder ($d_{\text{model}}=384$, 12 heads) consumes this memory via cross-attention and generates tokens autoregressively. Six prediction heads share the decoder's hidden state and provide structured output (token, type, motion-command, parameter-type, sign, per-position digit). A static grammar mask zeros structurally invalid token transitions in the legacy-token softmax at inference time.

The system has three stages (Figure 1): a frozen multi-modal encoder produces a memory sequence from the sensor window; a Transformer decoder generates the executing G-code line via cross-attention; and a grammar mask zeros structurally invalid softmax transitions in the legacy-token softmax at inference time. Encoder training is described in [16].

4.2. Sensor Encoder (Frozen)

The encoder (MM-DTAE-LSTM) is a multi-modal denoising autoencoder coupled to a temporal LSTM. Its input is seven modality groups: accelerometer (3-axis, 6 boards = 18 channels), gyroscope

(18), magnetometer (18), environmental temperature (6), ambient color (24), audio RMS (6), and motor current (8), totaling $C = 98$ channels. Per-modality projection maps each into a shared $d = 256$ space; learned modality-weighting gates [35,36] fuse them, a multi-head self-attention stack [20] captures cross-channel interactions, a denoising reconstruction objective [45] encourages noise-robust representations, and a two-layer LSTM [43,44] produces the temporal memory. The encoder has ≈ 5.1 M active parameters, trained for 30 epochs on the same file-level 5-fold splits, reaching 96.3% test-set accuracy on the nine-class operation-type task [16]. Training used AdamW ($\eta = 10^{-4}$), batch size 32, dropout 0.2, denoising noise $\sigma_{\text{noise}} = 0.1$, and early stopping on validation operation-class accuracy; per-fold checkpoint hashes are released. A representative test window appears in Figure 2.

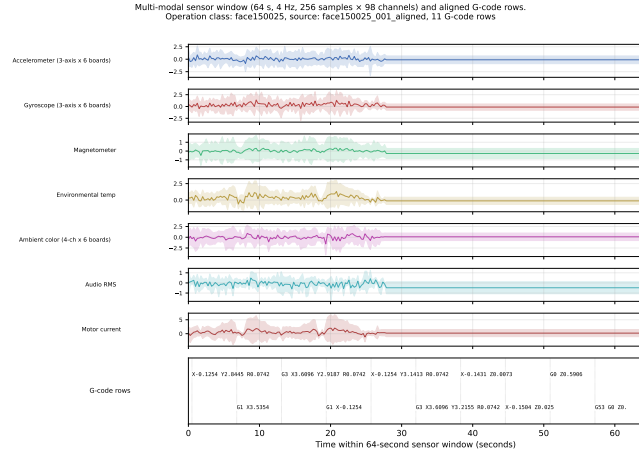


Figure 2. One 64-second test-split sensor window (`face150025_001_aligned`): seven modality groups (mean $\pm$ std per group) aligned in time against the 11 G-code rows that fired. Motor-current spikes align with G-code transitions. Visual analogue of the task formulation (Section 3.1).

4.3. Decoder Architecture

G-Code Decoder is an 8-layer Transformer decoder with $d_{\text{model}} = 384$ and $h = 12$ attention heads per layer. Each layer applies masked self-attention over the partial output, cross-attention over the encoder memory, a GELU feed-forward network, and pre-layer-norm residual connections [56], with dropout [57] of 0.1 after each sub-layer. The decoder has 31.28 M trainable parameters (18.9 M in the Transformer stack; full breakdown in Appendix C). With the frozen encoder (≈ 5.1 M active), total system size is ≈ 36.4 M parameters.

4.3.1. Multi-Head Structured Output

Six prediction heads share the decoder's final hidden state at each position. Each is a linear projection then softmax (six-position for the digit head), trained jointly with a weighted cross-entropy loss:

$$\mathcal{L}_{\text{decoder}} = \lambda_{\text{tok}} \mathcal{L}_{\text{tok}} + \lambda_{\text{type}} \mathcal{L}_{\text{type}} + \lambda_{\text{cmd}} \mathcal{L}_{\text{cmd}} + \lambda_{\text{param}} \mathcal{L}_{\text{param}} + \lambda_{\text{digit}} \mathcal{L}_{\text{digit}} + \lambda_{\text{sign}} \mathcal{L}_{\text{sign}}.$$

Default weights are $\lambda_{\text{tok}} = 3.0$ and all auxiliary $\lambda = 1.0$; the auxiliary heads regularize the shared representation toward token-type-aware features. A fold-1 Stage-1 sweep over $\lambda_{\text{tok}} \in \{1, 3, 5\}$ (Section 4.6) sets the default: $\lambda_{\text{tok}} = 1$ under-trains the token head (-4 pp TF token validation), $\lambda_{\text{tok}} = 5$ costs the categorical heads (-1 pp parameter-type and sign), and $\lambda_{\text{tok}} = 3$ is the validation-best compromise, a heuristic anchor rather than a 5-fold-optimized hyperparameter.

Legacy token head.

A vocabulary-sized softmax (2,418 classes) on the final hidden state; the primary inference output, grammar-masked (Section 4.3.2) before the softmax.

Type head.

A 4-way classifier over {SPECIAL, COMMAND, PARAM, NUMERIC}; the coarsest head, its accuracy upper-bounds the decoded sequence's structural sanity.

Motion-command head.

A 6-way classifier over {G0, G1, G2, G3, G53, M30} at COMMAND positions. G53 and M30 have very small support ($n \approx 6$ per fold, ≤ 30 pooled; Table 2), so their per-class F1 is underpowered (Section 8.2.0.1).

Parameter-type head.

A 10-way classifier over {X, Y, Z, I, J, K, R, F, S, P} at PARAM positions. Only X, Y, Z, R, and F have positive test-split support (Table 3); I, J, K, S, P are absent from the test data, their per-class entries regenerable from the released decoded-prediction JSONs.

Sign head.

A 3-way classifier over {POSITIVE, NEGATIVE, ABSENT} for the most recent axis-value's sign, supervising motion direction independently of the discretized value.

Per-position digit head.

A six-position 11-way classifier (digits 0–9 plus PAD) reconstructing each NUM token digit-by-digit over the corpus precision range (typically 0–2XXXX for X/Y), underlying the recoverability analysis of Section 6.4.

The categorical heads read this shared hidden state and therefore condition on the token stream; the evaluation-regime consequences (free-running collapse) are quantified in Section 3.4.

4.3.2. Grammar-Constrained Decoding

G-code follows a regular grammar: a line starts with an optional command (G/M) token, each address letter is followed by exactly one numeric value, and certain commands (e.g. G2, G3) require specific fields. We encode the local part as a static $|V| \times |V|$ binary transition mask on the legacy-token logits at each inference step (BOS $\rightarrow$ command/PARAM; PARAM $\rightarrow$ matching NUM; NUM $\rightarrow$ PARAM/command/EOS). The mask cuts the valid next-token set from $|V| = 2,418$ to 31.8 tokens on average (a static property of the released mask, regenerable from the tokenizer code). Implementation follows established runtime grammar-mask patterns [24–26,46–48] (Section 2.2).

Scope and limitations of the local mask.

The bigram mask does not encode command-state-dependent rules (e.g. R, valid only with G2/G3, must not follow G0), which require a finite-state grammar (FSM) over multi-token history. A separate inference-time FSM layer tracks the most recent move-command and forbids R/I/J after G0/G1 and any command/PARAM after M30, at autoregressive inference only (teacher-forced training already encodes correct structure); its full state specification is released with the artifacts.

FSM ablation.

With the FSM active, command-state (V1) violations drop from 329 (affecting 31% of baseline samples) to 0 (100% reduction) and from 333 (affecting 25% of with-shortcuts samples) to 11 (97% reduction), at a 5-fold accuracy cost within fold-to-fold noise (paired- t and Wilcoxon all $p > 0.4$). It eliminates the dominant grammar-violation class at no measurable accuracy cost; we adopt it as the default deployment-time decoding configuration. Violation counts are in `audit/grammar_violation_audit_baseline.json`; the paired-test statistics in `audit/fsm_vs_bigram_paired_stats.json`.

4.3.3. Scheduled Sampling

By default the decoder trains with teacher forcing; a scheduled-sampling regime [58] optionally replaces each input position with the model’s own previous prediction with probability p rising linearly from 0 to a tuned $p_{\max}$, reducing exposure bias. The sweep (Section 4.6) tested $p_{\max} \in \{0.1, 0.2, 0.5, 1.0\}$ against the $p_{\max} = 0$ default on a composite score (fold-1 validation token accuracy plus held-out test command/numeric/sequence accuracy). The headline $p_{\max} = 0.5$ lifted command accuracy +17 pp over $p_{\max} = 0$, at the cost of optimization stability at smaller numeric vocabularies (Section 6.6.2, where Design-B runs use $p_{\max} = 0$). Because the composite score touches held-out test metrics, this is a configuration choice, not an independent generalization estimate.

4.4. Training Configuration

The decoder is optimized with AdamW [59] ($\eta = 5 \times 10^{-5}$, weight decay 0.05) for up to 300 epochs with early stopping on validation token accuracy (patience 75), a 10-epoch linear warm-up, batch size 64, per-epoch shuffling, and gradient clipping at norm 1.0. The baseline trains *without* positional metadata (window_index, total_windows, source_file are not exposed; Section 6.6.1 ablates this). The 5-fold loss and token-accuracy curves decrease monotonically, plateauing near epoch 80–100, within the patience-75 budget; they are regenerable from the released training logs and figure scripts.

4.5. Encoder Information-Content Probe

Because the encoder is frozen, the decoder is upper-bounded by what its representation carries. We probe that bound with two-hidden-layer MLPs (hidden 256, ReLU, dropout 0.2) on the mean-pooled encoder memory for each of (command, axis-presence, motion-sign, operation-type), using the train/val/test protocol (MSE→RMSE for continuous targets).

Linear structure.

PCA on the 549×256 memory matrix (figures/encoder_memory_pca.pdf) confirms strong low-rank structure: the dominant subspace carries the class signal while fine numeric detail sits in the low-energy tail.

The probe results (audit/encoder_probe_v8.json) give three facts that bound the paper. First, *window-level signal exists*: operation type 0.846, axis presence ~ 0.87 – 0.93 (X/Y/Z), motion sign 0.80–0.86. Second, *per-row disambiguation does not*: a window holds a median of 29 rows (max 237) but one mean-pooled vector, so probe accuracy is capped near the modal-row-per-window ceiling (command reaches 0.77; the modal-row ceiling is ≈ 0.85), which motivates the full-window comparison (Section 6.6.3) and the class-prior confound (Section 7.1.0.2). Third, *numeric precision is lost*: X-value RMSE 0.97 approaches the within-window range (~ 3.0 normalized units), Y is 0.80, and Z is 0.10, so the encoder retains a window-level mean, not per-row precision. Row-level encoder pre-training is the natural remedy (Section 8).

4.6. Hyperparameter Sweep

The configuration was selected through a three-stage sweep on fold 1: Stage 1 (8 cells, learning rate, batch size, model dimension, loss-weighting); Stage 2 (12 cells, refining the winner over training techniques including scheduled sampling, label smoothing [60], and focal loss [61]); and Stage 3 (10 cells, architectural variants including encoder fine-tuning, pointer-network and regression-head outputs, cross-window attention, and multi-seed variance). The composite winner across all 30 cells (weighted-geometric-mean) was used for the 5-fold experiments of Section 6. The Stage-3 encoder-fine-tune pilot gave no meaningful lift (fold-1 command ~ 0.78 vs. frozen-encoder ~ 0.78) at $\sim 2\times$ cost, grounding the encoder-bottleneck claim of Section 4.5 as a measured-if-underpowered result. Because the sweep ran on fold 1 (the covariate-shift outlier of Section 8.2.0.2) and its composite score consulted fold-1 test metrics (Section 4.3.3), a selection-bias component attaches to fold 1 of

the headline mean; empirically fold 1 is the worst fold, so the bias is unlikely to inflate the mean, and the configuration *rank* is fold-1-composite-best rather than cross-fold-rank-stable. A cross-fold rank-stability check on the top-3 Stage-3 cells (a partial 5-cell, 2-fold replication) is on the artifact roadmap and not executed.

4.7. Architecture-Component Ablation Summary

Five architectural and training components are ablated, each pinned to the result section reporting its 5-fold effect: the grammar layer (Section 4.3.2), scheduled sampling (Section 4.3.3), positional metadata (Section 6.6.1), the target mode (Section 6.6.3), and the numeric vocabulary (Section 6.6.2). The six prediction heads are not ablated independently: the four auxiliary heads share the legacy-token head's encoder-memory feature as multi-task regularizers, so isolating each would require separate retrains. The categorical-vs-numeric factorization of Section 6.2 is the de facto head-level decomposition, with the categorical heads evaluated reference-conditioned (Section 3.4).

5. Experimental Setup

5.1. Dataset

The corpus is 120 aligned CSV files from a Bantam Tools desktop CNC milling machine [16]. Six distributed multi-sensor boards (frame_l2, frame_l3, frame_r2, spindle2, y_bed__3, y_bed__4) sample synchronously at 4 Hz. Each file carries accelerometer (3 axes $\times$ 6 boards = 18 channels), gyroscope (18), magnetometer (18), ambient color sensing (RGBA $\times$ 6 boards = 24), environmental temperature (6), audio root-mean-square (6), and motor-current draw (8), plus an aligned per-sample G-code line. Following the encoder paper's consistency audit [16] we drop proximity and pressure channels, retaining $C = 98$ sensor channels.

Sensor mounting and envelope-extraction pipeline.

The six boards (frame uprights, spindle housing, moving Y-bed) each carry an identical 15-channel inventory, plus 8 controller-side electrical channels. All channels are read at factory-rated sensitivities and envelope-extracted to the released 4 Hz rate by a single-pole IIR rectify-and-low-pass filter at $f_c = 2$ Hz, clock-aligned across boards, trading the kHz-band content of classical acoustic-emission monitoring [7] for a ~ 1.5 kB/s envelope; the single-pole rolloff (6 dB/octave, -3 dB at Nyquist) admits partially-aliased envelope content above 2 Hz, and the released stream is analyzed as-is. On-board pre-envelope rates are 1 kHz (audio), 200 Hz (MEMS accelerometer/gyroscope/magnetometer), and ≥ 100 Hz (temperature, RGBA color, motor current), with no mid-corpus re-calibration; firmware (src/firmware/envelope_extract/), chip-level part numbers, and electrical-channel mapping are released with the encoder paper [16].

The dataset spans nine machining operation classes: adaptive, adaptive150025, face, face150025, pocket, pocket150025, damageadaptive, damageface, damagepocket, crossing three motion strategies (face, pocket, adaptive) with three run conditions [16]: *standard* runs are air-cuts (spindle on, no material engagement), 150025 runs are active cuts engaging a UHMW-PE workpiece at 0.635 mm axial depth, and *damage** runs are air-cuts with one spindle drive band removed to induce spindle runout. The active (150025) programs post feeds of F13.3/F40.0 in/min versus the standard programs' F7.3/F22.0, with the posted spindle command constant at S12000 across the corpus. The six clean and parameter-variant classes hold 15–20 files each, the three damaged-spindle classes 4–5 each; the per-class inventory, distinct G-code-line counts, and aligned-sample counts are released in audit/operation_class_inventory.json.

Platform characterization and generalization risk.

The host is a Bantam Tools desktop CNC mill: cast-aluminum frame, ER11 collet spindle nominally to $\sim 28,000$ RPM, stepper-driven 3-axis gantry, no flood coolant, RS274/NGC-compatible dialect; active runs cut a single UHMW-PE workpiece dry with one 6.35 mm two-flute HSS end mill, and standard runs execute the same strategies as air-cuts [16]. Production VMCs of the Haas/DMG/Okuma class (cast-iron base, BT30/BT40 toolholders, servo drives, flood coolant) are expected to differ in vibration amplitude by 1–2 orders of magnitude, and further in spectral content, acoustic emission, and motor-current signature. The corpus therefore characterizes one low-amplitude, dry, stepper-driven platform; whether the factorization transfers to a production VMC is the open question of Section 8.4.

5.2. Preprocessing and Windowing

Sliding windows of $T = 256$ samples (64 seconds at 4 Hz) at stride 64 give ~ 550 windows per fold, each centered on continuous cutting and aligned to its overlapping G-code rows. The per-row target formulation (Section 3.3) emits one (W, g_r) pair per G-code row r that fires within window W , yielding $\sim 19,500$ training, $\sim 7,000$ validation, and $\sim 7,500$ test pairs per fold. At 4 Hz across the 98 retained float32 channels the on-line ingestion budget is ~ 1.5 kB/s, or ~ 0.13 GB per machine per 24-hour shift uncompressed.

Sampling-rate context.

The 4 Hz rate constrains what is recoverable. A row-execution-duration audit over the 120-file corpus (21,541 row-execution instances; `audit/gcode_row_durations.json`) shows the median G-code row executes in a single 0.25 s sample and 77.7% of instances occupy ≤ 1 sample period, against the ≥ 10 kHz of conventional acoustic-emission and vibration tool-condition monitoring [7]. Every recoverability result below should be read in that light; the full physical-aliasing discussion is in Section 8.2.

5.3. Five-Fold File-Level Stratified Cross-Validation

File-level stratified five-fold cross-validation prevents within-file leakage. Each operation class is split across folds in proportion to its frequency, so every fold covers all nine classes; within a fold, train and validation share a partition and test is the held-out partition.

Closed-vocabulary evaluation regime (load-bearing).

A G-code-line *coverage repair* step (Appendix A) swaps files between train and test partners until every test line also appears in training. This *constructs* a closed-vocabulary regime where every test line was seen verbatim in the same fold's training (97.6% of distinct test lines, 99.0% of instances; Section 8.1). The headline numbers are therefore within-vocabulary classification, not open-vocabulary recovery; the matched open-vocabulary number is the leave-one-class-out (LOCO) command accuracy of 0.213 ± 0.191 (Section 6.8).

Within-fold non-independence.

Overlapping windows (window 256, stride 64, 75% overlap between adjacent windows of a file) make adjacent within-file windows statistically non-independent, so the effective number of independent within-fold observations is below the raw window count, though file-level disjointness across folds holds. The covariate-shift audit (Section 8.2.0.2) and the bootstrap CIs on the 5-fold means therefore take the per-fold mean as the unit of analysis.

5.4. Baselines and Ablation Designs

We compare G-Code Decoder against the following baselines and ablations, all trained under identical preprocessing, splits, and hyperparameters:

1. **No-shortcuts baseline** (Section 6.2). Headline configuration; positional metadata withheld from the decoder.
2. **With-shortcuts ablation** (Section 6.6.1). Adds `window_index`, `total_windows`, and `source_file` hash as decoder inputs.
3. **Full-window target ablation** (Section 6.6.3). Multi-line per-window targets instead of per-row.
4. **Sensor-modality ablation** (Section 6.7). Zero each of the seven sensor groups at the encoder input and re-evaluate across five folds (audio-RMS at $n = 4$; one fold unrecoverable).
5. **Sequence-classifier (pattern-aware) ablation** (single-fold pilot; released artifacts). A jointly-trained sequence-classifier head predicts the reconstructed G-code line and biases the token distribution toward it.
6. **Vocabulary-precision ablation** (single-fold pilot; released artifacts). Coarser 2-digit numeric bucketing, $\sim 5\times$ smaller vocabulary.
7. **Noise-augmentation ablation** (released artifacts). Gaussian sensor noise, feature dropout, and token masking during training [22,23,57].
8. **Leave-one-class-out generalization** (Section 6.8). Train without each of the nine operation classes and evaluate on its held-out test samples.

5.5. Statistical Analysis

For each five-fold ablation we report the cross-fold mean and standard deviation, one-way ANOVA against the no-shortcuts baseline (SciPy [62]), and Cohen's d . Bootstrap 95% confidence intervals use 10,000 percentile resamples with the NumPy default RNG seeded at 42. Paired comparisons add the Hedges' small-sample-corrected effect size $g_z = J \cdot d_z$, $J = 1 - 3 / (4(n - 1) - 1)$; at $n = 5$ folds $J \approx 0.80$, so g_z runs $\sim 20\%$ below d_z throughout. Significance is marked $* p < 0.05$, $** p < 0.01$, $*** p < 0.001$. Reported $\pm$ values follow each audit script's native convention: NumPy population s.d. for the headline and sensor-ablation tables, `ddof=1` sample s.d. for the K -spectrum and no-numeric tables; the typical ~ 10 – 15% spread between conventions at $n = 5$ changes no significance call or deployment recommendation.

Inferential scope and a caution on $n = 5$.

The unit of analysis is the per-fold mean, so every parametric test is on $n = 5$ observations per group ($n = 4$ for audio-RMS, one fold unrecoverable). We treat all p -values as *descriptive* and the bootstrap 95% intervals as a smoothed, not coverage-calibrated, indication of cross-fold spread; conclusions rest on effect sizes large relative to that spread rather than on significance thresholds, with nonparametric (Wilcoxon) checks accompanying the paired tests where reported. Single-fold pilots are labeled as such and excluded from quantitative or deployment claims; the leave-one-class-out protocol (Section 6.8), though single-fold per holdout, is reported as a finite-population descriptive result over the nine-class corpus.

5.6. Software and Compute

The decoder is implemented in PyTorch and trained on NVIDIA RTX A6000 GPUs; preprocessing and analysis use scikit-learn [55] and SciPy [62]. The HistGradientBoosting non-neural baseline (Section 6.1) replaces XGBoost [63], whose available build was CUDA-only and failed on the CPU inference host (Appendix C). Sweeps were tracked with Weights and Biases; full code and configuration are released with the manuscript.

6. Results

6.1. Non-Neural Baseline Comparison

Two lightweight classifiers on the same mean-pooled encoder embedding the decoder consumes, a scikit-learn HistGradientBoosting (HGB) tree-boosters and the two-layer MLP probe of Section 4.5,

hit the most-common-class data-coverage ceiling on the binary fields (≈ 0.88 *has-X*, 0.99 *has-F*) but fall well short on multi-class command identity (HGB macro-F1 0.232, MLP 0.248), while HGB reaches macro-F1 0.86 on the one within-window-diverse field *has-Z*, so the pooled embedding carries some per-row Z signal. This comparison is a fold-1 reference on per-row targets, not a 5-fold result; the full comparison is released as `audit/nonneural_baselines.json`.

6.2. RQ1: Headline Accuracy and Per-Field Recoverability

Table 1 reports headline accuracies and macro precision/recall/F1 per head across five folds under teacher-forced (TF) and autoregressive (AR) evaluation. The categorical heads reach 0.9838 ± 0.0085 type accuracy, 0.8875 ± 0.0558 command accuracy, 0.9931 ± 0.0036 parameter-type accuracy, and 0.9888 ± 0.0063 pooled motion-sign accuracy; each categorical head is reported from the ground-truth-conditioned pass; under free-running generated-stream conditioning it collapses (command 0.483 ± 0.049 ; Section 3.4), so the categorical values are reference-conditioned quantities. The autoregressive-token heads diverge: token accuracy 0.7811 ± 0.0216 (TF) vs. 0.2148 ± 0.0685 (AR), numeric 0.5850 ± 0.0331 (TF) vs. 0.1038 ± 0.0322 (AR). Sequence exact-match is reported only under AR (per-fold {0.015, 0.000, 0.028, 0.056, 0.032}, mean 0.0263 ± 0.0185 ; `audit/ar_aggregate.json`); a TF sequence score is ill-defined, so the AR 0.0263 is the deployment-relevant figure, with the AR-drop mechanism in Section 7.3. The headline is seed-robust: four seeds span 0.0085 seed-to-seed, far inside the cross-fold spread (Appendix C).

We reserve *recovery* for the per-field categorical heads (command, parameter-type, sign) under reference-conditioned evaluation; the TF–AR gap on the token and numeric heads is the exposure-bias signature of autoregressive coupling (Section 3.4). End-to-end generative reconstruction is at floor (fewer than 3% of windows reproduced verbatim), so this paper claims per-field categorical recoverability, not toolpath reconstruction.

Robustness to the numeric-target patch.

A float-epsilon artifact at the finest decimal slots was corrected by a round-encoding retrain (`checkpoints/full_window_5fold_patched/`). Headline and categorical metrics are *robust* to it: patched vs. released 5-fold TF values are command 0.891/0.888, token 0.774/0.781, numeric 0.570/0.585, type 0.983/0.984, param-type 0.993/0.993, and AR sequence 0.029/0.026, every pooled/headline metric drifting ≤ 1.5 pp, so Table 1 uses the released checkpoint. The patched retrain is used only for the numeric tier (Section 6.4), where the correction concentrates, moving slot-5 accuracy from 0.77 to 0.99 while leaving the pooled numeric metric near-unchanged. The supervisory target_tokens are identical between runs; the corpus-weighted correction rate is 32.5% of numeric positions (30.1% terminal-zero $9 \rightarrow 0$ fixes). The historical iterative digitization (`slot[s] = int(dec_part * 10) % 10`) truncated IEEE 754 residuals: across $n = 79,563$ audited slot-5 positions it encoded 23.6% of CAM-authored terminal-zero values as slot-5 ‘9’ (worst on X, 32.8%; 0.0% on FP-clean R), while the round-to-scaled-integer rewrite (`src/miracle/model/digit_value_head.py`) yields zero corruptions (4.5% $\rightarrow$ 0.000%; `scripts/analysis/verify_digit_encoding_fix.py`; full characterization in [27]).

Three caveats govern these numbers. **First, they are closed-vocabulary, in-distribution** (coverage-repair regime, Section 5.3; 97.6%/99.0% verbatim test-line overlap), so the command accuracy of 0.8875 is within-vocabulary classification, *not* a deployment-ready open-vocabulary number. Second, the frozen encoder shared the file-level fold partitions, so the headline 0.888 carries a representation-reuse component atop the per-instruction sensor recovery claimed here, bounded two ways in Appendix A. Third, three companion numbers anchor it: the LOCO cross-class command accuracy of 0.213 ± 0.191 , the open-vocabulary floor (Section 6.8); the class-prior control (Section 7.1.0.2) for the within-class sensor-driven increment over the operation-class modal predictor; and the AR collapse described above.

Table 1. Headline test-set metrics for the G-Code Decoder under 5-fold file-level cross-validation, full-window targets, no positional metadata exposed. Mean $\pm$ std over 5 folds. Two evaluation regimes are reported: teacher-forced (TF), where each step receives the ground-truth previous token, and autoregressive (AR, greedy / beam-width 1), where the decoder consumes its own previous predictions. The categorical heads — type, command, parameter-type, sign — report a single reference-conditioned value (the AR column is marked ‘–’); measured free-running categorical accuracy is reported in the footnote. Only the autoregressive-token heads (token, sequence, numeric) carry distinct TF and AR values.

Head	TF Accuracy	AR Accuracy	Macro Precision	Macro F1
Token	0.7811 $\pm$ 0.0216	0.2148 $\pm$ 0.0685	0.4099 $\pm$ 0.0406	0.3899 $\pm$ 0.0405
Sequence [‡]	–	0.0263 $\pm$ 0.0185	–	–
Type	0.9838 $\pm$ 0.0085	–	0.9091 $\pm$ 0.0352	0.8833 $\pm$ 0.0451
Command	0.8875 $\pm$ 0.0558	–	0.9342 $\pm$ 0.0407	0.8790 $\pm$ 0.1084
Param-type	0.9931 $\pm$ 0.0036	–	0.9914 $\pm$ 0.0073	0.9857 $\pm$ 0.0130
Sign	0.9888 $\pm$ 0.0063	–	0.9658 $\pm$ 0.0215	0.9724 $\pm$ 0.0157
Numeric (overall digit)	0.5850 $\pm$ 0.0331	0.1038 $\pm$ 0.0322	–	–

Macro precision and F1 are unweighted means over all observed classes; the categorical heads carry a single (reference-conditioned) value. Numeric is the digit-head accuracy pooled across all six digit positions; the per-position bathtub is characterized in Section 6.4. The TF / AR gap on the token and numeric heads is the exposure-bias signature analyzed in Section 7.3; the categorical AR column is marked ‘–’ because those values are reference-conditioned; measured free-running categorical accuracy collapses (command 0.483 $\pm$ 0.049, type 0.551 $\pm$ 0.087, parameter-type 0.533 $\pm$ 0.046; `audit/ar_categorical_freerunning.json`). [‡] Sequence exact-match is reported only under AR; a sequence-level TF score is not separately well-defined (see Section 6.2).

The command head’s macro F1 (0.879 $\pm$ 0.108) is moderately below its raw accuracy (0.8875 $\pm$ 0.0558), reflecting the long-tailed class distribution: G1 carries $\sim$ 830 test instances per fold, G3 $\sim$ 315, G0 $\sim$ 65, and G53 and M30 $\sim$ 6 each. Recovery is strong across all six classes with no collapse (per-class F1 in Table 2). The lowest, G2 at 0.82, traces to bidirectional G1/G2 confusion in arc-rich windows (figures/confusion_matrix_command_normalized.pdf): its largest off-diagonal cells G1 $\rightarrow$ G2 ($n = 373$) and G2 $\rightarrow$ G1 ($n = 253$) are directionally symmetric, a local-pattern ambiguity between linear cuts and arcs, not a one-way bias. Per-class minimum detectable effects ($\sim$ 2.3 pp on G1, $\sim$ 8 pp on G2/G3, $\sim$ 19 pp on G0 at $\alpha = 0.05$, $\beta = 0.20$; `audit/extended_rigor_audits.json`, Section 8.2.0.1) bound the per-class column: smaller differences are not statistically distinguishable from sampling noise.

Table 2. Per-class precision / recall / F1 for the motion-command head, 5-fold mean $\pm$ std. Support is the approximate number of test instances per fold; per-class confidence intervals widen for the smallest-support classes, but no class collapses in 5-fold-mean F1 (the smallest-support classes G53 and M30, $n \approx 6$ /fold, nonetheless show wide per-fold recall variance and are underpowered).

Class	Precision	Recall	F1	Support
G0	0.946 $\pm$ 0.052	0.890 $\pm$ 0.142	0.917 $\pm$ 0.079	$\sim$ 65
G1	0.929 $\pm$ 0.024	0.886 $\pm$ 0.097	0.905 $\pm$ 0.049	$\sim$ 830
G2	0.811 $\pm$ 0.119	0.851 $\pm$ 0.039	0.823 $\pm$ 0.067	$\sim$ 400
G3	0.927 $\pm$ 0.072	0.913 $\pm$ 0.063	0.917 $\pm$ 0.043	$\sim$ 315
G53	1.000 $\pm$ 0.000	0.846 $\pm$ 0.327	0.876 $\pm$ 0.206	$\sim$ 6
M30	0.950 $\pm$ 0.112	0.769 $\pm$ 0.213	0.836 $\pm$ 0.150	$\sim$ 6

The parameter-type head’s macro F1 0.986 $\pm$ 0.013 closely tracks accuracy: the four well-supported axis letters (X, Y, Z, R) each reach F1 ≥ 0.989 , and thin-support F reaches F1 0.958 $\pm$ 0.042 on only $\sim$ 40 test instances. The full per-class precision/recall/F1 cells are regenerable from the released decoded-prediction JSONs.

6.3. Per-Axis Recoverability

For each axis letter we score *has-axis* (axis present in the predicted block), *sign* (motion sign when present: POSITIVE, NEGATIVE, ABSENT), and *value* MAE (mean absolute error where both predicted

and true blocks carry the axis). Table 4 reports these per axis; Table 3 gives per-field frequency, since near-zero-presence fields cannot be meaningfully evaluated.

Table 3. Per-field positive-support frequency in the V8 corpus. Accuracy numbers for a field with near-zero support (e.g. F, S, I, J in per_row mode) are dominated by the always-absent baseline and cannot be interpreted as evidence of sensor-driven recoverability. Source: direct count over the V8 preprocessed NPZ files, fold 1.

Field	per_row train	per_row test	full_window train	full_window test
X	90.4%	88.5%	100.0%	100.0%
Y	88.6%	86.6%	100.0%	100.0%
Z	35.8%	31.4%	29.7%	32.6%
F	0.4%	0.6%	18.8%	22.0%
S	0.0%	0.0%	0.0%	0.0%
R	15.2%	17.2%	88.1%	88.6%
I	0.0%	0.0%	0.0%	0.0%
J	0.0%	0.0%	0.0%	0.0%

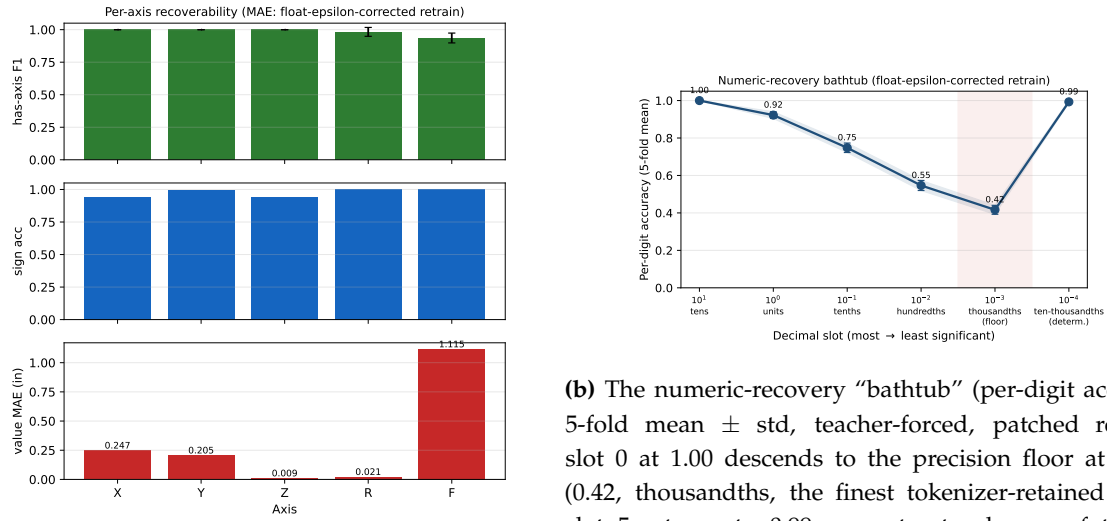
Bold cells indicate fields whose accuracy in Table 4 cannot be meaningfully evaluated due to insufficient positive support; recovery of these fields would require a corpus with non-trivial F/S/I/J coverage, beyond the present data.

Table 4. Per-axis G-code recoverability across 5 folds (mean $\pm$ std). **Value-MAE column re-derived on the float-epsilon-corrected retrain** (checkpoints/full_window_5fold_patched/; audit/patched_numeric_analysis.json); the categorical (has-axis, sign) columns are checkpoint-invariant. *has-axis Acc / F1* are the classification metrics for whether the axis is present in the decoded text; *sign* is the direction of motion (POSITIVE / NEGATIVE / ABSENT); *value MAE* is the mean absolute error of the numeric value, computed only on samples where both the predicted and true G-code carry the axis. The headline recoverability claim is restricted to the four well-supported axes **X, Y, Z, R**. The F row ($\dagger$) is reported for the full-window regime only, where F appears in 22% of windows; in the per-row regime F support is only 0.6% and is not evaluable (Table 3). I, J, S have effectively zero positive support in both regimes and are not evaluable. The per-axis sign-accuracy asymmetry (Y 0.997 vs. X 0.942 vs. Z 0.938) is prior-driven: Z is the only sign-bearing axis whose sign carries non-trivial classification load ($H_Z^{\text{sign}} = 1.04$ bits), whereas Y's sign is prior-deterministic ($H_Y^{\text{sign}} = 0.08$ bits) and X's is near-deterministic ($H_X^{\text{sign}} = 0.38$ bits); the per-axis sign source-entropy decomposition is developed in the companion limit paper [27], so the high Y/X sign accuracy reflects the prior, not sensor discrimination.

Axis	has-axis Acc	has-axis F1	Sign Acc	Value MAE	Presence Recall
X	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.942 $\pm$ 0.033	0.248 $\pm$ 0.140	1.000 $\pm$ 0.000
Y	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.997 $\pm$ 0.006	0.205 $\pm$ 0.080	1.000 $\pm$ 0.000
Z	1.000 $\pm$ 0.000	1.000 $\pm$ 0.000	0.938 $\pm$ 0.023	0.009 $\pm$ 0.006	1.000 $\pm$ 0.000
R	0.994 $\pm$ 0.012	0.983 $\pm$ 0.034	1.000 $\pm$ 0.000	0.021 $\pm$ 0.011	1.000 $\pm$ 0.000
F $\dagger$	0.960 $\pm$ 0.023	0.936 $\pm$ 0.038	1.000 $\pm$ 0.000	1.115 $\pm$ 0.350 (inch/min)	0.851 $\pm$ 0.093
S, I, J	insufficient support; see Table 3		—	—	—

X, Y, Z, R presence is fully recovered. The sign asymmetry is prior-driven, not a sensor-separability ranking: Y (0.997) and X (0.942) sign accuracy reflect their near-deterministic sign priors ($H_Y^{\text{sign}} = 0.08$, $H_X^{\text{sign}} = 0.38$ bits), while Z (0.938) is the only axis whose sign is genuinely balanced ($H_Z^{\text{sign}} = 1.04$ bits) and therefore the only one carrying real sensor-driven sign signal. Value MAE is small for Z (0.009) and R (0.021) and larger for X/Y (0.21–0.25), all in inches (the corpus's native unit; the platform runs in inch mode). Against the per-axis coordinate span over the corpus ($X \approx 3.8$, $Y \approx 3.8$, $Z \approx 0.7$, $R \approx 7.9$ in) these MAEs are $\approx 6.5\%$ and 5.4% of span on the large planar X/Y axes but only $\approx 1.2\%$ and 0.3% on the shallow-depth Z and arc-radius R axes — coordinate value is recovered to coarse resolution on X/Y and near-exactly on Z/R. These are the patched-retrain values (Z is 0.009 patched vs. 0.012 on the released checkpoint recomputed identically; an earlier reported 0.042 was a pre-patch analysis-pipeline artifact, not a checkpoint difference). The F-row MAE is on a different scale because feed-rate values themselves are larger (four distinct posted values, 7.3–40 in/min, in this corpus).

Four patterns emerge. First, axis-presence detection is uniformly high: X, Y, and Z reach perfect 5-fold has-axis accuracy (1.000 $\pm$ 0.000), R 0.994 $\pm$ 0.012, and F 0.960 $\pm$ 0.023 on its thin support. Second, motion-sign recovery is axis-asymmetric but prior-driven, not a sensor-separability ranking: Y-sign (0.997 $\pm$ 0.006) and X-sign (0.942 $\pm$ 0.033) ride near-deterministic priors ($H_Y^{\text{sign}} = 0.08$, $H_X^{\text{sign}} = 0.38$ bits; 3-way positive/negative/zero basis, audit/digit_entropy.json), while Z-sign



(a) Per-axis recoverability (5-fold mean): has-axis F1, motion-sign accuracy, and value MAE (inches). The Y-vs-Z sign asymmetry reflects the near-deterministic Y prior versus Z's balanced sign distribution [27].

(b) The numeric-recovery “bathtub” (per-digit accuracy, 5-fold mean $\pm$ std, teacher-forced, patched retrain): slot 0 at 1.00 descends to the precision floor at slot 4 (0.42, thousandths, the finest tokenizer-retained digit); slot 5 returns to 0.99 as a structural zero of the V8 tokenizer's 0.001 grid (discarding the ~ 3.31 -bit source digit), not information-bearing recovery [27]. Source: `audit/patched_numeric_analysis.json`.

Figure 3. The numeric tier: per-axis summary (left) and per-digit decomposition (right).

(0.938 $\pm$ 0.023), the only sensor-bearing balanced-distribution axis ($H_Z^{\text{sign}} = 1.04$ bits), scores lower as a harder sensor-driven binary classification, not weaker coupling. Third, the patched-retrain value MAE (`audit/patched_numeric_analysis.json`) is small for Z (0.009 patched; 0.012 on the released checkpoint) and arc-radius R (0.021) but larger for X (0.248) and Y (0.205), all in inches; the per-digit decomposition (Section 6.4) localizes this to middle digits, and the wide X fold spread (± 0.14) reflects large X-axis moves. Fourth, spindle (S) and arc parameters (I, J) have effectively zero positive support (Table 3) and cannot be meaningfully evaluated (Figure 3). The per-axis error distributions (`figures/numeric_error_histogram.pdf`) are bimodal: X/Y/Z errors heavy-tailed at 10^{-2} and 10^{-1} -inch magnitudes, R near exact-match, so the Table 4 MAE is a two-regime midpoint.

6.4. Numeric Digit Head Decomposition

The digit head achieves 0.5850 ± 0.0331 mean accuracy under teacher forcing, dropping to 0.1038 ± 0.0322 under autoregressive evaluation (Table 1). The per-position decomposition uses the TF condition on the float-epsilon-corrected retrain (`audit/patched_numeric_analysis.json`), aligning the entropy and accuracy axes at the finest digits. The pooled 5-fold per-slot accuracy is bathtub-shaped (Figure 3b): slot 0 (10^1 , tens) 0.9999; slot 1 (10^0 , units) 0.9223 ± 0.0180 ; slot 2 (tenths) 0.7478 ± 0.0253 ; slot 3 (hundredths) 0.5465 ± 0.0269 ; and slot 4 (10^{-3} , thousandths) bottoms out at 0.4165 ± 0.0239 , the *true precision floor*, since the thousandths is the finest digit the V8 tokenizer retains (the CAM source carries a fourth decimal the 0.001 grid discards [27]). Slot 5 (10^{-4} , ten-thousandths) returns to 0.9931 ± 0.0067 because the V8 tokenizer quantizes X/Y/Z onto a 0.001 grid ($p_X = p_Y = p_Z = 10^{-3}$; Appendix A), making the tokenized target's ten-thousandths digit structurally near-zero entropy (0.31 bits), deterministically reproduced rather than information-bearing. This slot-5 “upturn” is a tokenizer representation choice, not a source or sensor limit: the companion limit paper [27] shows R, quantized on a 0.0001 grid, retains its ten-thousandths digit. On the corrected encoding the operation-class stratification sits at slot 4 (the thousandths), not slot 5: facing operations carry 2.10 bits there while pocketing (3.32) and adaptive (3.31) sit at the CAM-interpolation ceiling, and the raw-source slot-5 is near-uniform (≈ 3.3 bits) for every class (`audit/per_slot_per_opclass_entropy.json`; the earlier slot-5 stratification was a float-epsilon encoding artifact, retracted with the full account in [27]).

The decoder therefore behaves as a *coarse-resolution coordinate recoverer*: it reproduces the physically- or geometrically-constrained high-order digits and the deterministic trailing digit, bounded at a near-uniform-noise ceiling ($\approx \log_2 10$ bits) at the thousandths place where the CAM postprocessor's interpolation residuals dominate. The digit head tracks the corpus's per-(axis, slot) source-entropy ceiling [27] (corrected pooled $n = 6$ Pearson -0.89 ; per-(axis,slot) $n = 30$ Pearson -0.83).

Per-axis sign recovery (Z is the only sensor-bearing sign axis).

The motion-sign head's 0.9888 headline accuracy (Table 1) is dominated by prior-deterministic axes ($\sim 99\%$ positive Y, $\sim 93\%$ positive X) and should not be read as a sensor-recovery figure; only Z carries non-trivial sign load ($H_Z^{\text{sign}} = 1.04$ bits), splitting evenly between positive rapids above the stock and negative cuts into it. The deployment-relevant per-axis sign accuracies are the X/Y/Z means in Table 4 (0.94/1.00/0.94 5-fold), Z carrying essentially all the discriminative work; the X/Y priors would fail on a workpiece of different orientation or origin [27].

6.5. Per-Operation-Class Recoverability

Autoregressive predictions stratify sharply by operation class from a single mechanistic cause, not a per-class capability gradient. *face* ($n = 144$ test windows) reaches token 0.56 / numeric 0.40 against a pooled AR baseline of 0.215/0.104, while every other class falls into a narrow 0.09–0.25 token / 0.01–0.15 numeric band; the most-frequent non-facing class, *pocket150025* ($n = 124$), sits at floor-level 0.09/0.01 despite ample test support. This is the mode-collapse signature of Section 7.3: the corpus-modal AR output the decoder converges to *is* a facing-pattern toolpath, so *face* scores by coincidental agreement with it, not sensor-driven discrimination, and any other toolpath signature scores near floor regardless of support. The per-class *numeric* story is thus constant-mode-collapse with one outlier, not evidence that *face* is easier to decode; the headline categorical recoverability (Section 6.2) remains the central per-class claim (Figure 5a, all nine classes).

6.6. RQ2: Metadata vs. Sensor and Per-Row vs. Full-Window

RQ2 asks (i) whether positional metadata inflates accuracy beyond what sensor data alone provides, and (ii) whether the per-row target formulation introduces a row-identification ambiguity that full-window targets resolve (Table 5; the target-mode contrast itself is in Section 6.6.3).

Table 5. Ablation matrix. Each ablation row reports both teacher-forced (TF) and autoregressive (AR, beam-width-1 greedy) test accuracy where the metric depends on the regime. The categorical heads (type, command, param-type, sign) carry a single value because their metrics are computed from the ground-truth-conditioned (teacher-forced) pass in both regimes; free-running autoregressive categorical accuracy is not separately measured. The pattern-aware row is a fold-1 pilot (released artifacts); the sensor-modality effect is summarized here and broken out per modality in Table 7 (5-fold; audio-RMS $n = 4$). Numbers from `outputs/decoder20260511/audit/ar_aggregate.json` (5-fold AR) and the per-fold `metrics.json` (5-fold TF), commit db246b0.

Configuration	Token (TF / AR)	Sequence (AR)	Type	Command	Param-type	Numeric (TF / AR)
Baseline (no shortcuts, full-window)	$0.781 \pm 0.022 / 0.215 \pm 0.069$	0.026 ± 0.019	0.984 ± 0.009	0.888 ± 0.056	0.993 ± 0.004	$0.585 \pm 0.033 / 0.104 \pm 0.032$
+ positional metadata	$0.754 \pm 0.033 / 0.290 \pm 0.016$	0.102 ± 0.022	0.982 ± 0.011	0.882 ± 0.056	0.993 ± 0.004	$0.532 \pm 0.054 / 0.184 \pm 0.017$
+ sequence classifier (pattern-aware, fold-1)	$0.694 / -$	0.010	0.975	0.536	0.952	$0.392 / -$
— any single sensor modality (5-fold; rms $n = 4$)	command 0.449–0.484 (uniform -40 to -44 pp; between-modality ANOVA $p = 0.998$); token ~ 0.74 ; type / param-type within ~ 1 –4 pp of baseline. Full per-modality breakdown: Table 7.					

The TF / AR gap is the exposure-bias signature analyzed in Section 7.3: token and numeric heads consume their own previous predictions under AR and accumulate compounding errors over the $\sim 1,000$ -token full-window sequence. The categorical metrics, computed from the reference-conditioned pass, do not register the AR collapse; measured free-running categorical accuracy falls to 0.483 on command (`audit/ar_categorical_freerunning.json`). The + positional metadata ablation reverses sign across regimes: under TF it is mildly harmful or neutral, under AR it produces +7.5 pp on token, +8.0 pp on numeric, and +7.6 pp on sequence, with $\sim 4\times$ tighter fold-to-fold variance on the token head. Section 6.6.1 interprets this as mitigation of autoregressive mode collapse rather than a positional shortcut.

6.6.1. Positional-Metadata Ablation

Re-training the decoder *with* positional metadata exposed (window index, total-window count, source-file hash) gives the effect sizes in Table 5 (the + *positional metadata* row), which one-way ANOVA across the 5 folds factorizes cleanly.

Categorical heads are recoverable from sensors alone and unaffected by positional metadata: $\Delta \approx 0$ on command/type/param-type/sign ($F < 0.05$; evaluations reference-conditioned, Section 3.4), and the with-shortcuts 5-fold command mean 0.882 ± 0.056 is statistically indistinguishable from the no-shortcuts 0.8875 ± 0.0558 baseline. At $n = 5$ folds per group these are descriptive null effects, not confirmed equivalences.

Autoregressive heads depend on the regime. Under TF positional metadata gives no lift and is mildly harmful (-2.7 pp token, -5.3 pp numeric); under autoregressive deployment-true evaluation it reverses to token $\Delta = +7.5$ pp ($F = 4.5$), sequence $\Delta = +7.6$ pp ($F = 27$), and numeric $\Delta = +8.0$ pp ($F = 19$). At $n = 5$ folds every p -value is descriptive ($p \sim 10^{-3}$ sequence and numeric, $p \sim 0.07$ token); the claim rests on the consistent $+7.5$ – 8.0 pp increments. The mechanism is mode-collapse mitigation: the window index anchors each window and breaks the corpus-modal trajectory on a fraction of test samples, a deployment-time disambiguation signal, not a sweep-time shortcut (Section 7.3). Baseline cross-fold spread intervals (95%, not coverage-calibrated at $n = 5$) are in `audit/bootstrap_ci.json`.

Multiple-comparisons correction on the drilled grid.

The per-class ANOVA grid (Section 5.5) runs 840 tests over 8 configurations; Benjamini–Hochberg FDR ($\alpha = 0.05$) flags 457/840 and Holm–Bonferroni 167/840, but each is an $n = 5$ ANOVA, so this is a *robustness sweep*, not 457 established effects. The signal is carried by a few large-effect ablations (noise-augmentation per-class command F1 drops of 0.3–0.7 for G0/G1/G2/G3; uniform command-head degradation across all seven modality ablations); the correct reading is the sign and size of Δ (`audit/anova_per_class_results.json`), not any adjusted p -value threshold at cells with $n_{\text{support}} \leq 50$ (Section 8.2.0.1).

6.6.2. No-Numeric Retraining: Decomposing the Autoregressive Collapse

Roughly half of every G-code line’s positions are coordinate *values* (Section 6.4), each a hard selection from the $\sim 2,400$ -entry numeric vocabulary, and a single mispredicted value desynchronizes the rest of the autoregressive decode. To test whether the AR collapse of Section 7.3 is *driven* by numeric tokens, we retrain the decoder with every coordinate value collapsed to a single placeholder (24-entry structural vocabulary). Masking numerics post-hoc is contaminated, since mispredicted values desynchronize surrounding structural tokens before the mask applies (structural-only 0.317 vs. unmasked 0.215); removing them from the prediction problem is the clean test. Encoder, 5-fold split, and hyperparameters are unchanged, except scheduled sampling is disabled (the placeholder vocabulary destabilizes optimization, Section 4.3.3); validation token accuracy converges at ~ 0.96 (last-25-epoch s.d. < 0.01).

Table 6 reports the comparison. Under TF the no-numeric decoder reaches structural token 0.927 ± 0.024 (headline 0.960 ± 0.020) and structural sequence exact-match 0.068 ± 0.017 (headline 0.213 ± 0.048): a real but modest cost. Under AR the two effects decompose: structural per-token improves $+8.8$ pp (0.405 ± 0.143 vs. 0.317 ± 0.116 ; 5/5 folds, paired $t = 3.56$, $p = 0.024$, Wilcoxon $p = 0.06$), but structural sequence exact-match is unchanged (0.066 ± 0.014 vs. 0.058 ± 0.039 , 2/5 folds), so window-level recovery does not improve. The collapse decomposes into (a) numeric-token desynchronization, eliminated here, and (b) a dominant intrinsic mode-collapse of the structural token stream, which the ablation does not affect. A matched-methodology control ($K = 2,418$ T3.1 cell, same schedule) shows the numeric-vocabulary collapse adds no lift beyond the schedule change T3.1 already absorbs, downgrading the standalone “ ~ 9 pp numeric-desynchronization” framing.

Table 6. No-numeric retraining ablation: structural-skeleton recovery of the headline decoder versus a decoder retrained with every coordinate value collapsed to a single placeholder token. *Structural* restricts the metric to command-code and axis-letter positions, excluding numeric/placeholder positions. 5-fold mean $\pm$ std, full-window targets, under teacher-forced (TF) and autoregressive (AR) evaluation. Under TF the no-numeric decoder is modestly weaker on structure (-3 pp per-token, -15 pp per-window) — the cost of collapsing the lexical variety of the numeric vocabulary. Under AR the two effects decompose: per-token structural accuracy improves $+8.8$ pp (paired across folds, 5/5 folds positive, $p=0.024$, Wilcoxon $p=0.06$) — the numeric-desynchronization component of the collapse, whose standalone contribution the matched-methodology control of Section 6.6.2 bounds — but per-window structural exact-match is unchanged, identifying intrinsic structural mode-collapse as the dominant window-level mechanism.

Metric	Headline decoder		No-numeric decoder	
	TF	AR	TF	AR
Structural token accuracy	0.960 ± 0.020	0.317 ± 0.116	0.927 ± 0.024	0.405 ± 0.143
Structural sequence exact-match	0.213 ± 0.048	0.058 ± 0.039	0.068 ± 0.017	0.066 ± 0.014

Vocabulary-cardinality spectrum and matched-methodology control.

A matched-methodology K -spectrum control ($K \in \{24, 69, 335, 2,418\}$ under $SS=0/dw=0$) makes $K=2,418$ (T3.1) the directionally strongest variant on every metric (5/5 folds positive on AR struct token, $+15.2$ pp over the headline; full confound resolution and retraction in Section 8.3), with the paired AR-token contrast Holm $p=0.071$ (n.s. at $n=5$) and AR-sequence Holm $p=0.006$. AR struct token is monotonically non-decreasing in K (0.405, 0.415, 0.429, 0.470) and AR struct sequence exact-match broadly increasing (0.066, 0.051, 0.081, 0.128, minor non-monotonicity at $K=69$), and the five-variant omnibus is significant on both (AR struct token $F(4, 16)=3.42$, $p=0.033$, partial $\eta^2=0.46$, Friedman exact $p=0.0018$; AR struct sequence $F(4, 16)=7.38$, $p=0.0014$, $\eta^2=0.65$, Friedman exact $p=0.019$). Within the matched family $K=335$ falls -4.1 pp behind $K=2,418$ on AR struct token (paired $t(4)=-0.85$, $p=0.44$, n.s.); per-fold values and effect sizes are in `audit/k_spectrum_compare.json` and `audit/k_spectrum_stats.json`.

6.6.3. Target-Mode Ablation (Per-Row vs. Full-Window)

The per-row formulation (Section 3.3) leaves the decoder without a within-window row-identification signal: every row shares the same encoder memory. Full-window targeting instead supervises on the entire multi-line G-code in the window, so row r is predicted conditional on rows $1 \dots r-1$.

With architecture, hyperparameters, and data fixed, command accuracy moves from per-row 0.499 to full-window 0.888, a $+38.9$ pp gain measured reference-conditioned (Section 3.4); part of the full-window advantage reflects that mode's access to ground-truth within-window history under teacher forcing (Section 7.1.0.2). In per-row mode the decoder cannot disambiguate which of ~ 60 candidate rows (median 29) within a 64-second window it must emit; full-window targeting gives every row a unique within-window position signal, restoring identifiability for the command head.

On the autoregressive-token heads the contrast is regime-dependent: teacher-forced, full-window is higher on token and numeric; autoregressive, the difference shrinks as exposure-bias errors compound over the longer sequences. Full-window is therefore preferred only for categorical recovery; for end-to-end sequence reproduction, per-row is comparable under AR and much shorter to infer.

6.7. RQ3: Sensor-Modality Ablation

For each of seven sensor modality groups (accelerometer, gyroscope, magnetometer, environmental, color, audio-RMS, electrical) we zero the corresponding channels at the encoder input and evaluate the decoder without retraining, isolating its marginal contribution *under the encoder's*

existing weight allocation. This is a lower bound on each modality's value; the system-level test is the encoder retraining of Section 6.7.1. Audio-RMS is evaluated at $n = 4$ (one fold's ablation run was unrecoverable) and the other six at $n = 5$; given the uniform ~ 40 – 44 pp collapse the verdicts do not hinge on the missing fold.

Uniform collapse, not modality differentiation.

Every modality is critical to the command head, at a statistically uniform magnitude. Zeroing any single modality drops command accuracy from the full-modality baseline 0.888 ± 0.056 to the 0.449 – 0.484 band (accelerometer 0.484 , environmental 0.474 , magnetometer 0.465 , gyroscope 0.462 , rms 0.460 , color 0.457 , electrical 0.449 ; all ± 0.05 – 0.11), a uniform -40 to -44 pp collapse. The ANOVA across the seven groups is *not* significant ($F = 0.07$, $p = 0.998$; `audit/sensor_anova_7modality.json`), so the modalities are not separable at the present fold count, while every modality versus the baseline is significant under a Welch test ($p < 0.001$, t from -7.1 to -11.5). **No single modality is shown more deletable than another:** this confirms joint necessity but cannot rank importance. A TOST + BF_{01} equivalence check on the seven means ($\Delta \leq 0.035$ at a ± 0.020 margin) is *inconclusive* on several pairs at $n = 5$, at the resolution limit of a five-block test. Token, type, and parameter-type heads are more robust (token ~ 4 pp, type < 1 pp, parameter-type ~ 1.5 pp across every ablation). Table 7 and Figure 4a report the per-modality numbers.

Table 7. Sensor-modality ablation (mean $\pm$ std; $n = 5$ folds per modality, except audio-RMS at $n = 4$ — one fold's ablation run was unrecoverable). Each row zeros the named sensor channels at the encoder input and re-evaluates the decoder; the encoder is *not* retrained, so this is a lower bound on modality value. A one-way ANOVA across the seven zeroed-modality groups on command accuracy is not significant ($F = 0.07$, $p = 0.998$): the seven modalities are statistically indistinguishable from one another. Every modality versus the full-modality baseline is significant (Welch t , $p < 0.001$ in all seven cases), with a uniform ~ 40 – 44 pp command-accuracy collapse. Token, type, and parameter-type heads stay within ~ 4 pp of baseline for every removal.

Modality removed	Token	Type	Command	Param-type	Numeric
<i>baseline (5-fold)</i>	0.781 ± 0.022	0.984 ± 0.009	0.888 ± 0.056	0.993 ± 0.004	0.585 ± 0.033
zero accelerometer	0.743 ± 0.010	0.979 ± 0.010	0.484 ± 0.096	0.978 ± 0.002	0.471 ± 0.008
zero color	0.741 ± 0.010	0.977 ± 0.008	0.457 ± 0.108	0.977 ± 0.004	0.471 ± 0.009
zero electrical	0.741 ± 0.009	0.976 ± 0.009	0.449 ± 0.086	0.978 ± 0.002	0.465 ± 0.009
zero environmental	0.743 ± 0.010	0.977 ± 0.008	0.474 ± 0.077	0.978 ± 0.002	0.470 ± 0.011
zero gyroscope	0.741 ± 0.009	0.979 ± 0.005	0.462 ± 0.049	0.978 ± 0.003	0.469 ± 0.006
zero magnetometer	0.735 ± 0.018	0.975 ± 0.010	0.465 ± 0.084	0.978 ± 0.002	0.466 ± 0.017
zero rms	0.739 ± 0.010	0.978 ± 0.007	0.460 ± 0.088	0.978 ± 0.002	0.465 ± 0.010

The uniform command collapse with a non-significant between-modality ANOVA ($p = 0.998$) is the key result: the inference-time zeroing test confirms all modalities are jointly necessary but cannot rank them. The genuine modality-importance ranking is provided by the leave-one-modality-out encoder retraining (Section 6.7.1).

Interpretation.

The encoder's learned modality-gating routes command-discrimination signal through every modality at once; zeroing any one degenerates the representation to one sufficient for the coarse (type, param-type) heads but not the command head, so the frozen-encoder "joint necessity" reading is a gating artifact and the genuine ranking needs the retraining of Section 6.7.1. It does not match the encoder paper's SHAP ranking (gyroscope/color/electrical) because the two answer different questions: inference-time joint necessity under the existing encoder versus training-time operation-class signal.

6.7.1. Per-Modality Encoder Retraining (Leave-One-Modality-Out)

Because the zeroing ablation cannot rank modalities (Section 6.7), the deployment-relevant question is whether the encoder, retrained without a modality, reallocates capacity onto the survivors. We retrain the multi-modal denoising encoder from scratch on the 98-feature base with each modality removed, train a fresh six-head decoder on the resulting frozen encoder, and score all seven cells against a retrained full-stack baseline on the identical held-out test set; a deterministic coverage-repair ordering and per-cell channel-identity gate guarantee a byte-identical split across exclusions. The zeroing study's "environmental" group reduces to temperature on the 98-feature base (pressure and proximity already excluded as low-activity channels), so the LOMO "Temperature" row is its counterpart.

No measurable single-modality effect.

Retraining the encoder without any single modality produces no measurable change in any decoder head. The full-stack baseline reaches command accuracy 0.904 ± 0.039 , and every removal lands within ± 0.013 (electrical largest at -0.013 ± 0.015). The fold-blocked ANOVA across the seven conditions is non-significant ($F = 0.91$, $p = 0.499$), and the Holm-corrected paired t -test against zero rejects none (0/7 at $\alpha = 0.05$). Token and numeric heads behave the same (paired effects within ± 0.011 token, ± 0.022 numeric). Table 8 and Figure 4b chart the paired deltas against the frozen-encoder collapse of Figure 4a. Magnetometer is closest to significant (adjusted $p = 0.41$, Hedges' $g_z \approx -0.94$), a medium-large paired effect that fails significance at $n = 5$ but is informative. Per-fold Δ , Holm-corrected p , and effect sizes are in `audit/lomo_modality_attribution.json`.

Table 8. Leave-one-modality-out (LOMO) encoder retraining, full-window, five-fold. Each row: the encoder retrained from scratch with that modality's channels removed, then a fresh decoder trained on it (headline recipe). *Command* is the five-fold mean $\pm$ s.d. Δ *Command* is the *fold-paired* effect — the mean over folds of $\text{command}(m, f) - \text{command}(\text{baseline}, f)$ — which removes the large fold-to-fold variance. p_{Holm} is the Holm-corrected one-sample t -test of those per-fold deltas against zero. Contrast with the inference-time zeroing of Table 7, which holds a frozen encoder fixed.

Modality removed	Command	Token	Numeric	Δ Command (paired)	p_{Holm}
<i>Full stack (baseline)</i>	0.904 ± 0.039	0.774 ± 0.024	0.569 ± 0.050	—	—
Accelerometer	0.903 ± 0.037	0.769 ± 0.018	0.559 ± 0.039	-0.001 ± 0.005	1.000
Gyroscope	0.905 ± 0.030	0.772 ± 0.019	0.569 ± 0.041	$+0.000 \pm 0.010$	1.000
Magnetometer	0.896 ± 0.033	0.761 ± 0.018	0.547 ± 0.040	-0.008 ± 0.007	0.407
Color	0.904 ± 0.044	0.773 ± 0.018	0.569 ± 0.038	-0.001 ± 0.013	1.000
Temperature	0.904 ± 0.038	0.775 ± 0.023	0.571 ± 0.049	-0.000 ± 0.015	1.000
Audio-RMS	0.905 ± 0.039	0.779 ± 0.022	0.582 ± 0.043	$+0.000 \pm 0.017$	1.000
Electrical	0.891 ± 0.051	0.776 ± 0.024	0.576 ± 0.050	-0.013 ± 0.015	0.710

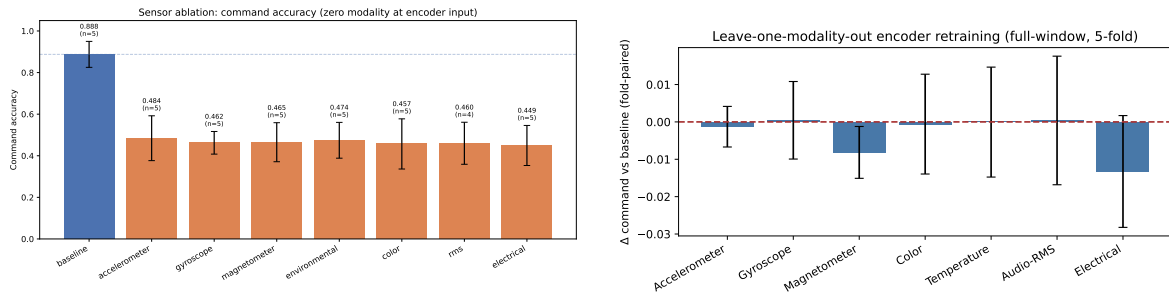
Fold-blocked one-way ANOVA across the seven LOMO conditions (on the per-fold deltas): $F = 0.91$, $p = 0.499$. Per-modality paired t -test vs. baseline (Holm-corrected): 0/7 significant at $\alpha = 0.05$. Coverage: 40/40 cells.

Interpretation.

The -42 pp frozen-encoder collapse of Section 6.7 reduces to ≤ 1.3 pp under encoder retraining (none significant), confirming the gating artifact: retraining reallocates capacity onto the remaining channels. The deployment-relevant statement upgrades from "the full multi-modal stack is required" to "no single modality is individually load-bearing".

Orthogonal probes (summary).

Two probes corroborate the LOMO main effect. The physical-sensor-unit axis (which board a deployment could ship without) is the LOSO study: per-board removals cost ≤ 2.6 pp (fold-blocked ANOVA $p = 0.415$; 0/6 Holm-significant; `y_bed_4` indistinguishable from baseline at $p_{\text{Holm}} = 0.975$; `audit/lomo_sensor_attribution.json`), and the nested 6×6 interior is flat within ± 0.022 ($p =$



(a) Frozen encoder, inference-time zeroing: removing any single modality drops command accuracy by 40–44 pp, and the ~ 4 pp span across modalities does not differentiate importance (5-fold mean $\pm$ std; audio-RMS at $n = 4$; Table 7).

(b) Encoder retraining (LOMO): per-modality fold-paired Δ command vs. the retrained baseline; every modality within ± 0.013 , none Holm-significant (fold-blocked ANOVA $F = 0.91$, $p = 0.499$).

Figure 4. The sensor-redundancy contrast: the frozen-encoder –42 pp uniform collapse (left) versus the null effect under per-modality encoder retraining (right); the gap is the encoder’s re-allocation capacity, not the modalities’ information content.

0.896; 0/36; audit/lobo_nested_attribution.json). No single-fold interaction pilot overturns the modality main effect or the headline. The three nulls (LOMO $p = 0.499$, LOSO $p = 0.415$, nested $p = 0.896$) align: the redundancy is dense rather than a marginal-average effect; the joint deployment statement is consolidated in Section 7.2.0.1.

6.8. RQ4: Cross-Class Generalization (Leave-One-Class-Out)

For each of the nine operation classes (adaptive, adaptive150025, face, face150025, pocket, pocket150025, damageadaptive, damageface, damagepocket) we train the decoder without that class and evaluate on its held-out samples, testing whether the decoder generalizes across operations or memorizes per-class shortcuts.

Each holdout removes every file of the held-out class from the fold 1 partitions, re-stratifies the remaining 8-class partition at the headline 80/20 split, and tests on all held-out-class files (fold 1 is also the covariate-shift outlier of Section 8.2.0.2, so the LOCO numbers are conditional on this partition choice). The coverage-repair step is deliberately *not* re-run, leaving the open-vocabulary residual exposed. Hyperparameters are reused verbatim and each holdout trained from scratch.

Across all nine LOCO holdouts (one fold per holdout, fold-1 file split): token accuracy mean 0.501 (min 0.29, median 0.53, max 0.61); sequence accuracy mean 0.001 (range [0, 0.005]); command accuracy mean 0.213 (min 0.000, median 0.21, max 0.678); numeric accuracy mean 0.152 (min 0.000, median 0.16, max 0.402). Type and parameter-type are more robust (means 0.883 and 0.945; spreads ± 0.061 and ± 0.036 , subject to the same single-fold caveat below).

Two scope caveats govern these numbers. The nine operation classes are the corpus population, not a sample, so we report the finite-population descriptive distribution, not between-class parametric/bootstrap CIs, and make no inferential claim about a wider class population. The per-holdout numbers are single-fold (file-split fold 1 only), so the open-vocabulary command mean 0.213 is a single-fold result and the 9-class spread $\sigma = 0.191$ reflects between-class heterogeneity, not fold-to-fold sampling.

Relative to the in-distribution 5-fold baseline (0.781/0.026/0.888/0.585), LOCO drops ~ 28 pp on token, ~ 3 pp on sequence, ~ 67 pp on command (mean), and ~ 43 pp on numeric. This is a descriptive gap, not a two-sample test: file-stratified CV and class enumeration are different experimental designs, so the released Welch and Mann–Whitney statistics (audit/loco_significance.json: Welch $t = -9.2$, $p \approx 2.5 \times 10^{-6}$; MWU $p = 0.003$) are descriptive corroboration only. The evidence is the per-holdout collapse itself (command-mean gap 0.675); the 5-fold and 9-class per-cell ranges do not overlap on command or numeric.

Per-holdout variance and the damagepocket outlier.

The nine per-holdout command accuracies span $[0.000, 0.678]$ with $\sigma = 0.191$. Eight cluster in $[0.000, 0.273]$; one, damagepocket, reaches command 0.678, token 0.548, numeric 0.402, because it shares most of its motion vocabulary (predominantly G1 linear cuts with modest G3 finishing arcs) with the retained pocket and pocket150025 variants. damageadaptive (command 0.000) and damageface (command 0.094) do not benefit, because adaptive and face motion patterns differ more between damaged and non-damaged variants than the pocket variants do. Cross-class generalization therefore succeeds when the held-out class shares motion vocabulary with retained classes and collapses otherwise; the median (0.21) reflects typical behavior better than the mean.

Per-class LOCO command F1.

Under an unseen motion vocabulary the model defaults to the modal G1 (Section 7.3), and the collapse spans every command class: G0 0.084 ± 0.136 , G1 0.243 ± 0.277 (modal; variance driven by the damagepocket outlier), G2 0.044 ± 0.107 , G3 0.083 ± 0.134 , G53/M30 0.000 at $n \leq 5$ (audit/loco_per_class_9holdouts.json). The heads degrade asymmetrically: type and parameter-type survive LOCO ($-10/-5$ pp), so the encoder preserves gross instruction structure for a novel class, while command and numeric require the in-distribution pattern. The deployment-realistic claim is therefore bounded by these LOCO numbers, not the 5-fold headline (Section 8.1), and is scoped to in-distribution post-hoc audit, closed-world over the G-code operation classes observed at training time; open-world / novel-operation-class deployment is out of scope and deferred to future cross-platform replication (Section 8). Within the closed-world envelope, tamper-detection operating points are developed in the companion security study (in preparation; Section 7.4); outside it the system is not a usable detector (LOCO command-swap FPR 0.52, sign-flip and feed-edit at or below chance; audit/threat_model_tamper_loco.json). Figure 5b charts the per-holdout, per-head breakdown.

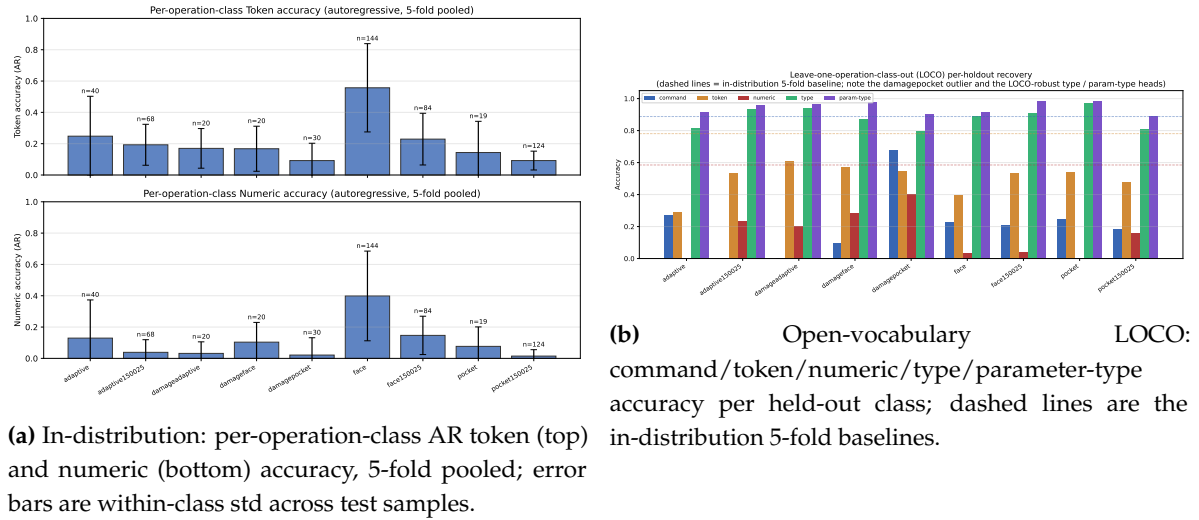


Figure 5. Per-class stratification of autoregressive recovery, in-distribution (left) and under leave-one-class-out (right).

6.9. End-to-End Seq2Seq Baseline: What the Frozen Encoder + Grammar Mask Buy

The headline stack (frozen MM-DTAE encoder, grammar-constrained autoregressive decoder, six-head structured output) uses no individually novel component; we compare it against a from-scratch vanilla Transformer encoder-decoder (8.64 M parameters, same order as the frozen encoder's ≈ 5.1 M active: 4-layer encoder + 4-layer decoder, $d_{\text{model}} = 256$, 8 heads) trained directly on (sensor window $\rightarrow$ G-code token sequence) with cross-entropy, sharing the V8 vocabulary, full-window target, file-level 5-fold CV, and metrics but with no frozen

encoder, grammar mask, six-head output, scheduled sampling, or auxiliary losses (code: `scripts/evaluation/run_seq2seq_baseline.py`, `src/miracle/model/seq2seq_baseline.py`; per-fold results released under `seq2seq_baseline/fold_{1..5}/`). Teacher-forced command accuracy is architecture-invariant (0.877 ± 0.031 baseline vs. headline 0.888 ± 0.056), and the apparent +76.6 pp command / +77.9 pp parameter-type AR gap is a metric-construction artifact: the baseline reads command identity off the AR token stream and collapses to the most-frequent-class prior 0.122 ± 0.051 (parameter-type read: 0.214 ± 0.082 against the reference-conditioned 0.993), whereas the headline command head is evaluated reference-conditioned at 0.888 (free-running it collapses to 0.483; Section 3.4), so the raw gap compares conditioning regimes; like-for-like free-running, the headline command head (0.483) still exceeds the baseline's AR-derived command (0.122), and on the AR token stream the two are within a fold (token 0.154 vs. 0.215; numeric 0.100 vs. 0.104). The baseline's TF numeric accuracy 0.754 ± 0.053 exceeds the headline's bucketized digit-decomposition output 0.585 ± 0.033 by +16.9 pp, a future-work item that does not translate to an AR-mode advantage (AR numeric 0.100 vs. 0.104). The ~77 pp AR categorical gap therefore reflects the conditioning-regime difference, not autoregressive-decoder superiority; the categorical heads carry the audit-mode signal when a reference stream (the controller log) is available to condition on (Section 7.2.0.1).

7. Discussion

7.1. What is recoverable, and the class-prior boundary

What is recoverable, and why.

Recoverability factorizes into three structural tiers (Section 6.2, Table 4). *Categorical* structure (command and axis identity) is strongest: the sensor pathway determines the motion class and the axes involved independently of file position, and the metadata ablation (Section 6.6.1) confirms positional features add nothing. *Motion sign* is recovered with an axis asymmetry, Y near-perfect and X and Z lower, on the prior-driven account of Section 6.3. *Numeric precision* is the limiting tier, and the limit is information-theoretic: loss concentrates at the middle (fine-value) digit positions (Section 6.4), because a 64-second window observes only relative motion, not absolute machine coordinates. Recovering the latter requires positional context (deliberately withheld) or longer-range autoregressive context; the per-row deployment mode provides neither, and under the full-window headline the numeric ceiling persists through aliasing and source entropy (Section 6.4).

Is categorical recovery sensor-driven or operation-class structure?

The frozen encoder was trained for nine-class operation classification on the *same* file-level fold partitions used here (Appendix A), and its memory is low-rank and linearly separates the operation classes (Section 4.5). The headline command accuracy could therefore be operation-class structure re-expressed at the command level. A class-prior control (`audit/confound_class_prior.json`; `scripts/analysis/confound_class_prior.py`) tests this: at every COMMAND-token position of the full-window predictions, the decoder is compared against a *class-prior* predictor emitting the modal command of the window's operation class (modal on the fold's training split) and against a within-window modal predictor.

The decoder's command accuracy at command positions is 0.795 ± 0.121 (5-fold), versus a class-prior baseline of 0.572 ± 0.004 and a within-window-modal baseline of 0.587 ± 0.003 . The 0.795 is measured only at COMMAND-token positions of the full-window predictions, a different position subset and aggregation than the 0.8875 head accuracy of Table 1 (consistent measurements, not a discrepancy; cf. Section 8.3.0.1). Among history-free predictors, the operation-class modal is the strictest legitimate sensor-independent null: each test file is unseen, so a train-only per-file modal reduces to it.

The decoder's 0.795, however, is measured under teacher forcing with ground-truth within-window token history; a sensor-free null granted the same history, a per-operation-class first-order Markov command model fitted on each fold's training split, reaches 0.662 ± 0.004 at the same positions (audit/markov_command_null.json), so the increment over a history-matched sensor-free null is $\approx +13$ pp rather than +22 pp.

The +22 pp increment over the class-prior null is a 4-of-5-fold effect, not a uniform one. On fold 1 the decoder (0.552) is at or below its own class prior (0.568, a -0.016 delta; fold 1 is the covariate-shift outlier of Section 8.2.0.2), while folds 2–5 sit +0.27 to +0.30 above the prior (0.85–0.86 decoder vs. ~ 0.57). The paired-by-fold mean increment is +0.222 (per-fold deltas $\{-0.02, +0.28, +0.27, +0.30, +0.28\}$, 4/5 positive), driven by the four non-outlier folds (the paired mean remains $\geq +0.18$ after removal of any two folds). At $n = 5$ we report it as an effect size, not a p -value, since no five-point test can confirm an increment whose sign flips on one fold. A percentile bootstrap over the five deltas gives a 95% CI of approximately $[+0.10, +0.29]$; jackknife means are +0.20 to +0.21 dropping any of folds 2–5 and +0.28 dropping the fold-1 outlier (folds 2–5 mean +0.282), so the fragility is a leave-one-fold effect, not a bootstrap-edge one. The magnitude is an *upper* bound on the sensor-driven within-class signal.

A majority of the headline command signal (roughly 0.57 of the 0.79) is therefore operation-class structure the encoder already encodes, the representational counterpart of the leave-one-class-out collapse (Section 6.8). On top of that prior the decoder adds the within-class increment on the $97.4\% \pm 1.4\%$ of windows containing more than one distinct command; its accuracy on those command-heterogeneous windows, 0.794 ± 0.122 , matches its pooled accuracy, so it is not echoing a per-window stereotype.

This increment is an UPPER bound (closed-vocabulary regime, frozen-encoder representation reuse, ground-truth-history conditioning), but unchanged under a test-derived class-modal null. The 0.572 null is the per-operation-class modal estimated on each fold's training split; re-deriving the same class-modal predictor on the test split, scored on the same all-command-position basis, reaches only 0.576 ± 0.003 (5-fold; audit/class_conditional_modal_5fold.json), so the +22 pp increment is not an artifact of estimating the class modal from training data. (A first-command-position-only class-conditional modal reaches ~ 0.81 , audit/class_conditional_modal.json, an easier estimand on a single position per window, not a like-for-like null for the decoder's all-position accuracy.)

7.2. Sensor-stack redundancy and deployment recommendations

Sensor-modality importance and deployment implications.

A cost-reduced post-hoc audit installation is viable on this platform. Among single-board removals the point estimates rank y_bed_4 cheapest ($\Delta \approx 0$, $p_{\text{Holm}} = 0.975$) and spindle2 costliest (-0.026), an illustrative ordering of non-significant $n = 5$ estimates, not a confirmed ranking. The frozen-encoder inference-time ablation establishes *joint necessity* (40–44 pp uniform collapse, between-modality ANOVA $p = 0.998$). The encoder-retraining LOMO complement (Section 6.7.1) shows no single modality is individually load-bearing (paired Δ command ≤ 0.013 ; $F = 0.91$, $p = 0.499$), so the frozen-encoder collapse is a gating artifact. The LOSO study (audit/lomo_sensor_attribution.json) shows any single physical sensor board costs ≤ 2.6 pp (largest spindle2 -0.026 ; y_bed_4 ≈ 0 at $p_{\text{Holm}} = 0.975$), and the nested 6×6 interior (audit/lomo_nested_attribution.json) is flat within ± 0.022 ($F = 0.694$, $p = 0.896$). Redundancy is dense: ≤ 1.3 pp expected cost per modality, ≤ 2.6 pp per board.

When to use this decoder vs. alternative approaches.

The decoder suits three offline patterns: post-hoc audit of contested runs (primary; the posture of Section 7.4), end-of-shift process verification as a human-in-the-loop triage aid (alert-budget operating

points in the companion security study, in preparation; Section 7.4), and localization of session-level anomaly flags [6,34] to specific G-code lines. In autoregressive deployment the per-row target mode is preferable to full-window: per-row predictions are short and independent across rows, and so robust to the full-window mode-collapse of Section 7.3; the command-head cost of the per-row mode is the 0.499-versus-0.888 target-mode contrast of Section 6.6.3. The decoder is *not* a stand-alone controller-log replacement (no exact-line reproduction): numeric precision is too low for coordinate reconstruction, though command/axis/sign analysis is reliable within the trained toolpath envelope.

7.3. Autoregressive Mode Collapse on Full-Window Targets

The autoregressive decoder converges toward a corpus-modal facing-pattern toolpath. Across the 549-sample 5-fold test split it produces 407 distinct strings (vs. 459 for ground truth), and the modal predicted string appears in 9.5% of windows vs. 2.9% for the modal ground-truth, a $3.3\times$ over-representation (audit/ar_output_diversity.json). The collapse is therefore *partial*: over-concentration on the modal facing pattern, not a single canonical output. It drives the autoregressive-token-accuracy drop of Section 6.2; the categorical metrics are unaffected by construction (computed from the reference-conditioned pass; Section 3.4). The target-length correlation (Pearson $r \approx -0.15$) is weak and treated as corroborating only, not load-bearing. The collapse is class-dependent: face-class windows dominate the modal-output fraction because the modal sequence is itself a facing-pattern toolpath (figures/class_conditional_modal_freq.pdf). Widening the beam does not recover the exposure-bias loss: on per-row targets beam-3/beam-5 lift token and numeric accuracy by < 1 pp over greedy (measured on the released checkpoints; not persisted as a released artifact) (TF $\rightarrow$ AR-greedy gap ~ 10 pp token, ~ 12 pp numeric), and on full-window targets the TF $\rightarrow$ AR gap is -53 pp token and -45 pp numeric, so beam search was not rerun there. Section 7.2.0.2 recommends per-row mode at deployment time on this basis.

The beam-width-one decodings of fold 1's first three full-window samples illustrate the mechanism: samples 1 and 3 receive essentially the same token stream despite different targets, while sample 2 coincides with the corpus-modal sequence and scores near-perfectly by accident. The pairwise-overlap analysis (figures/mode_collapse_heatmap.pdf) shows within-class prediction pairs 60–80% identical vs. $\sim 20\%$ across-class, dropping $0.20 \rightarrow 0.17$ with positional metadata.

7.4. Security Implications and the Companion Threat-Model Study

The per-field recoverability characterized here bounds what a sensor-side integrity monitor can and cannot see: categorical substitutions (command, axis, motion-sign) leave a recoverable trace, while fine-grained numeric-coordinate and feed-rate edits do not (Section 6.3; gross deviations are assessed in the companion study). A companion security study (in preparation) develops the corresponding tamper-detection operating points, alert-budget analysis, and adaptive-adversary bounds under the same closed-world, single-platform scope; those results are reported separately so that this paper's claims rest on recoverability alone.

8. Limitations and Future Work

8.1. Dataset and Platform Limitations

The most consequential dataset limitation is the near-absence of variation in feed rate (F), spindle speed (S), and arc parameters (I/J). Table 3 reports each field's positive support: F appears in $< 1\%$ of per-row samples, S is essentially absent from both modes, and I and J never appear (arcs use R notation). The Table 4 accuracies for these fields are therefore artifacts of near-zero baseline support, not sensor-driven recovery. The corpus is also dominated by frequently-recurring lines, G1 strongly dominant with a long tail: each fold contains of order 10^3 distinct lines but only a handful of motion-command classes (per-fold counts in audit/train_test_gcode_overlap.json), and per-class

metrics macro-average over that tail, so G3, M30, and arc parameters would benefit from more diverse data.

Active runs use one workpiece material (UHMW-PE) at one axial depth on one Bantam Tools mill (cast-aluminum frame, stepper drive, no flood coolant, dry cutting). 69 of the 120 runs are air-cuts (55 standard, 14 damaged-spindle), so only 51 runs engage material; testing material-class generalization would require a multi-material corpus (aluminum, steel, Delrin). *Cross-platform generalization is the larger open question.* A production VMC differs from this platform along every sensing axis quantified in Section 5.1 (“Platform characterization”), and no result here has been replicated on one. Until cross-platform replication (Section 8.4, item (d)), every recoverability claim, deployment recommendation, and detectable-attack-class guarantee is conditional on this single platform/material envelope: a characterization study, not a portable deployment claim. The AR mode-collapse mechanism, the structured-vs-numeric factorization, the class-prior decomposition, and the seq2seq architectural-contribution scoping are mechanism findings expected to hold under any low-rate multi-modal stack and Transformer decoder; the headline numbers, the 6-sensor 7-modality inventory, the UHMW-PE signature, the 4 Hz rate, and the 9-class taxonomy are platform-specific and expected to transfer only with adjustment.

Verbatim train/test line overlap (closed-vocabulary regime).

The file-level 5-fold splits guarantee file-level train/test disjointness, but the coverage-repair procedure (Section 5.3; Appendix A) ensures by design that every test-partition G-code line was seen during that fold’s training. The measured line-level overlap (audit/train_test_gcode_overlap.json) is 97.6% of distinct test lines and 99.0% of test line-instances in per_row mode (5-fold average), and 97.9%/99.1% in full_window mode. The 5-fold experiments therefore evaluate mapping a sensor signal to an in-vocabulary G-code line, not generating novel lines; the open-vocabulary evidence is the LOCO protocol of Section 6.8, under which command accuracy degrades from 0.888 to 0.213 ± 0.191 and numeric from 0.585 to 0.152.

8.2. Methodological and Statistical Limitations

The encoder was trained on the same fold splits as the decoder, and its objective (nine-class operation classification) includes no row-level G-code supervision; both may limit its preservation of fine-grained per-row structure. Retraining with row-level supervision would quantify the residual encoder-side contribution, the principal open direction (Section 8.4). The 256-sample window length was inherited from the encoder paper [16] for consistent encoder inputs; a window/stride sweep requires retraining the encoder per (window, stride) pair and is not separable on the present frozen encoder.

The 4 Hz rate is not marginal. The row-execution-duration audit (21,541 instances; audit/gcode_row_durations.json) shows the median row executing in a single 0.25 s sample and **77.7% of instances occupying ≤ 1 sample period** ($83.1\% \leq 0.5$ s, $86.3\% \leq 1$ s), so most instructions are sub-sample events the stack cannot resolve as distinct transients. The observable signal is the lower-frequency cutting-regime envelope, which is why categorical structure is recoverable while per-instruction numeric precision is not; the mechanism is physical, not only the sequence head’s exposure bias. The 4 Hz rate is nevertheless *not* the sole numeric bottleneck: in the per-row deployment mode (Section 3.3) a within-window row-identification ambiguity additionally caps numeric recovery independently of sample rate, while the full-window headline removes that ambiguity yet still meets the entropy-and-aliasing ceiling (Section 6.4); a higher-rate stack addresses the aliasing component, the autoregressive sequence head the per-row component. We have not characterized the per-modality PSD/SNR of the 4 Hz envelope stream against a co-recorded raw stream, so the numeric-tier ceiling cannot be partitioned between envelope spectral loss and sub-sample aliasing; the ceiling is a property of the present stack, not the architecture. A higher-rate next-iteration stack (contact microphone

≥ 44.1 kHz; axis encoders, IMU, and Hall-effect spindle sensors at ≥ 1 kHz) could close the numeric tier and the AR-mode gap while leaving categorical recovery unchanged.

Statistical power.

Per-class command-head metrics rest on small per-class supports (power-analysis basis, per-row test counts; the full-window per-fold counts of Table 2 differ in basis): G0 ($n=86$), G1 ($n=1,095$), G2 ($n=475$), G3 ($n=395$), none ($n=5,886$). A power analysis (`audit/extended_rigor_audits.json`, key `rec6_power_analysis`) computes each class's minimum detectable effect (MDE) at $\alpha=0.05$, $\beta=0.20$: G0 (baseline ~ 0.40) MDE ≈ 18.8 pp, G1 (~ 0.94) ≈ 2.3 pp, G2 (~ 0.50) ≈ 8.0 pp, G3 (~ 0.30) ≈ 8.4 pp, none (~ 0.95) ≈ 0.9 pp, and param-types X, Y, Z, R ≤ 1.5 pp. Differences below these sizes are not distinguishable from sampling noise; “not significant” on the long-tail classes (G0, G3) often means *underpowered*, not *equivalent*.

Covariate shift.

A per-field train-vs-test positive-support audit (`audit/extended_rigor_audits.json`, keys `rec7_covariate_shift_*`) flags only modest shifts (< 6 pp) on already-low-support fields: per-row fold-1 Z -4.4 pp; full-window fold-1 F $+3.2$ pp and fold-5 F -5.3 pp; none elsewhere. These contribute to the cross-fold standard deviation in Table 4 but do not change the per-axis ranking.

8.3. Evaluation and Per-File Limitations

Per-file variance.

Slicing the 5-fold predictions by source file ($n = 91$ of the 120 corpus files receive full-window test predictions; the remaining 29 contribute no test windows; released `full_window_5fold` JSONs) shows per-file command accuracy concentrated, not heavy-tailed (0.953 ± 0.088 ; 10th percentile 0.846; span $[0.571, 1.000]$; parameter-type 0.991), so categorical-head success is not carried by a few memorizable files. AR token accuracy shows a wide per-file spread ($[0.052, 0.869]$, 0.302 ± 0.283), tracking the per-operation-class breakdown (Section 6.5). The unweighted per-file mean (0.953) differs from the per-fold-mean headline (0.8875 ± 0.0558) by aggregation, not a prediction discrepancy. The entropy–accuracy inverse relation of Section 6.4 reproduces per-file (figures/per_file_fingerprint.pdf; [27]).

Each operation-class file is a single execution, so the metrics of Section 6 conflate cross-condition variance with cross-trial variance; a multi-replicate corpus would support variance decomposition, the present one does not. Per-row autoregressive decoding on the RTX A6000 runs 30–60 ms per G-code line at the median row length (5–15 tokens, $K=2,418$, FSM-masked greedy; measured on the development host, not persisted as an artifact), within the 250 ms inter-sample headroom of the 4 Hz cadence; full-window mode ($\sim 1,000$ – $1,400$ tokens per sample) costs proportionally more and is not real-time on this hardware. Beam search does not close the AR gap (Section 7.3); its deployment-cost profile and non-autoregressive variants remain unevaluated.

K-spectrum methodology confound: resolved.

The four-point vocabulary-cardinality sweep originally compared the headline $K=2,418$ row ($SS=0.5$, $dw=1.0$) against three smaller- K Design B variants ($SS=0$, $dw=0$), jointly contrasting cardinality and training schedule. A methodology-matched $K=2,418$ control under $SS=0/dw=0$ (T3.1; Section 6.6.2) resolves it: under matched methodology $K=2,418$ is the highest-performing variant on every metric (AR struct token 0.470 ± 0.034 vs. $K=335$'s 0.429 ± 0.123 and the headline's 0.317 ± 0.116). The earlier $+11.1$ pp $K=335$ advantage was the SS/dw schedule change, not cardinality; the $K=335$ “sweet spot” framing is accordingly retracted. The deployment-relevant configuration for

AR structural recovery is $K=2,418$ with $SS=0/dw=0$, subject to cross-platform replication before it carries forward (Section 8.4).

TF-numeric baseline reversal (future-work item, not a deployment recommendation).

The seq2seq baseline (Section 6.9) attains higher teacher-forced numeric accuracy (0.754 ± 0.053) than the headline six-head digit-decomposition output (0.585 ± 0.033), a gap attributable to the bucketization that makes the grammar mask tractable in the legacy-token head (Section 3.2). Under autoregressive inference the two are within a fold (0.100 vs. 0.104), so the reversal is a numeric-head future-work item, not an AR-mode deployment advantage. A non-autoregressive CTC-head pilot and a single-legacy-head Transformer-decoder pilot remain unexecuted on the artifact roadmap.

8.4. Future Directions

Four directions remain open, each an open problem rather than a scheduled experiment.

(a) *Continuous-parameter data-collection campaign.* A factorial corpus varying feed rate (F), spindle speed (S), depth of cut, and material at ≥ 100 Hz (to lift the 4-Hz aliasing of 77.7% of rows) would populate the F/S/I/J fields the present 4-Hz corpus cannot evaluate. The first experiment is a $3 \times 3 \times 3$ (feed, speed, depth) full factorial with a higher-rate front end, scoped to recover feed-rate substitutions at the resolution the adversarial scenarios of Section 7.4 require.

(b) *Encoder retraining with row-level supervision.* The fold-1 last-LSTM-layer fine-tune pilot (Section 4.6) is within fold-1 noise and underpowered to settle the encoder-bottleneck question; the deliverable is a five-fold retrained encoder under matched preprocessing, checkpoints released, isolating the encoder-side residual.

(c) *Window/stride re-optimization.* Requires the encoder retraining of (b); not separable on the present frozen encoder. Planned coverage is a 4×4 (window, stride) grid retrained per cell with the row-level-supervised encoder.

(d) *Cross-platform validation.* A production-class VMC (cast-iron base, servo drive, flood coolant) and a second material class (aluminum or steel) with a matching sensor stack, running the present file-level five-fold protocol on the new corpus to yield the cross-platform transfer matrix for the categorical-vs-numeric factorization.

9. Conclusion

This study characterizes which components of an executing G-code block are recoverable from multi-modal CNC sensor data, using G-Code Decoder, a six-head Transformer decoder on a frozen multi-modal denoising encoder with per-row and full-window supervised targets and grammar-constrained inference, under five-fold file-level cross-validation. Three findings follow, each detailed in the section cited:

- (1) **Per-block categorical recovery** (command identity, axis presence, motion direction) is sensor-driven under reference-conditioned evaluation (Section 6.2); measured free-running categorical accuracy collapses to 0.483 (audit/ar_categorical_freerunning.json). The class-prior-controlled within-class increment over the history-free operation-class modal (Section 7.1.0.2) is unchanged under a test-derived class-modal null and reduces to $\approx +13$ pp against a history-matched Markov null. On an *unseen* operation class the increment does not transfer: command recovery collapses to the open-vocabulary floor of 0.213, below the class prior (Section 6.8).
- (2) **End-to-end token/numeric reproduction** is at floor (autoregressive whole-sequence exact-match 0.026). Two mechanisms bound it: exposure-bias mode-collapse in the autoregressive head (Section 7.3) and 4-Hz temporal aliasing of 77.7% of rows (Section 8.2), acting on top of an information-theoretic floor the companion limit paper [27] characterizes. On the float-epsilon-corrected retrain the per-digit precision floor sits at the *thousandths* (10^{-3} inch, slot

accuracy 0.42), the finest digit the V8 tokenizer retains (the CAM source carries a fourth decimal the 0.001 grid discards [27]); the ten-thousandths digit is a structural zero of the V8 tokenizer's 0.001 grid, reproduced trivially rather than information-bearing (Section 6.4). Generative coordinate reconstruction is unsupported on this stack. The K -spectrum retraction and matched-methodology T3.1 control are in Section 8.3.

- (3) **Sensor-stack redundancy.** Under the frozen encoder, inference-time modality removal collapses command accuracy uniformly ($p = 0.998$ between modalities; Section 6.7). Encoder-retraining ablations (LOMO and LOSO; Section 7.2.0.1) show no single sensing modality or sensor unit is individually decisive ($\Delta \leq 2.6$ pp), enabling a cost-reduced post-hoc audit installation.

A from-scratch Transformer seq2seq baseline (Section 6.9) matches the headline command accuracy under teacher forcing but collapses under autoregression, so the headline numbers stand and the architectural contribution of the frozen encoder, grammar-constrained decoder, and structured heads is sharpened to AR deployment-mode inference, the regime the post-hoc audit posture requires. Closing the gap to full reconstruction is a sequence-head problem first; whether the encoder is co-responsible cannot be settled by the present single-fold, underpowered last-LSTM-layer fine-tune pilot (Section 4.6) and requires full 5-fold encoder retraining with row-level supervision across the multi-modal projection and attention stack.

Standard exposure-bias remedies [64–68], a higher sample rate, encoder retraining with row-level supervision, and richer continuous-parameter data are the principal directions for the TRL 5–6 cross-platform target (Section 8). Cross-platform replication on a production-class VMC is the prerequisite for any deployment claim outside the single desktop CNC platform characterized here. The present **forensic-support post-hoc audit** posture is the boundary within which the results hold, and none of these directions upgrades it to real-time SOC alerting, which requires a structurally different sensing stack and an open-vocabulary recoverability regime the present corpus does not support.

Appendix A Specification Addenda

Coverage-repair split procedure.

The file-level stratified k -fold splits are post-processed into the closed-vocabulary regime of Section 5.3 by swapping files until every test line also appears in its fold's training partner. Each swap exchanges the test file contributing the most test-only lines with the training file that minimally perturbs the per-class fold proportions (alphabetical tie-break; swap budget 10; residual test-only lines recorded as a novel-vocabulary diagnostic); it converges in 0–3 swaps per fold, repaired fold sizes within one file per class of the stratified target. The per-fold assignment is released as `audit/file_fold_assignment.csv`.

Encoder specification addendum.

The encoder architecture and training recipe are stated in Section 4.2. The representation-reuse caveat of Section 6.2 is quantified two ways: the worst-case class-label-leakage bound caps the leakage component of command accuracy at 0.572 (so the +22 pp class-prior-controlled increment of Section 7.1.0.2 is leakage-immune), and the leave-decoder-test-files-out LOMO-baseline check bounds the empirical reuse effect at $|\Delta| \leq 0.016$ (0.904 ± 0.039 vs. headline 0.888 ± 0.056).

Grammar tokenization addendum.

The per-axis quantization precisions are $p_X = p_Y = p_Z = p_P = p_Q = 10^{-3}$, $p_I = p_J = p_K = p_R = p_E = 10^{-4}$, $p_F = 1.0$ (in/min), and $p_S = 10.0$ (rpm); F and S are excluded from the first-order PID structural grammar. The NUM_a_b bucket index is $b_a(v) = \text{sgn}(v) \cdot \text{round}(|v|/p_a)$ (rounding, as the worked example $X1.4919 \rightarrow \text{NUM}_X_{1492}$ requires), with the sign carried by the separate sign head

(Section 4.3.1) and zero mapping to $b = 0$ on every axis. Per-axis bucket counts (1,048 NUM_X, 1,086 NUM_Y, 231 NUM_Z, with smaller bucket sets for the remaining axes, summing to 2,418) are determined empirically as the cardinality of the bucket-index set on the training corpus; any test-time index outside the trained range falls back to UNK (closed-vocabulary regime, Section 5.3). The numeric-tier results are computed on the float-epsilon-corrected round-to-scaled-integer encoding; the categorical/token headline metrics drift ≤ 1.5 pp between encodings (Section 6.2).

Appendix B Relationship to the Companion Contributions and Data Availability

This study is one of four related contributions. (i) The encoder paper [16] establishes a multi-modal denoising sensor encoder and its operation-class representational competence (96.3% nine-class accuracy). (ii) A grammar paper [17] defines the hybrid G-code tokenization. (iii) *The present paper* addresses per-row, instruction-level G-code *recoverability*: the per-field structured-output evaluation, the metadata-versus-sensor and per-row-versus-full-window ablations, the leave-one-class-out generalization study, the exposure-bias / mode-collapse characterization, and the tamper-detection scoping. It takes the encoder and grammar as fixed and asks what row-level instruction information survives the sensor→latent map, re-deriving neither. (iv) A companion limit paper [27] develops the decoder-agnostic information-theoretic and physical recoverability limit — the per-(axis, slot, operation-class) source entropy of the CNC toolpath corpus, the per-axis sign and per-digit entropy decompositions, the float-epsilon numeric-target-encoding audit, and the 4 Hz sub-Nyquist floor — that bounds the per-field recoverability characterized here. Beyond these four, a security companion (in preparation) develops the tamper-detection operating points deferred from Section 7.4. Stripped of the companion contributions, the standalone methodological novelty is modest (a frozen embedding, a Transformer decoder, a runtime grammar mask); the contribution is the characterization itself, including the negative results and the explicit in-distribution / open-vocabulary boundary. The encoder and grammar specification addenda (Section 4.2; Appendix A) keep the work independently assessable and reproducible.

Data and code availability.

The 120-file aligned corpus, the per-fold preprocessed arrays, the decoded prediction JSONs, and all audit artifacts referenced here are released as described in the Data Availability Statement below. Nothing required to reproduce this manuscript's results depends on the companion contributions. Raw confusion-matrix counts and other per-cell diagnostics are recomputable from the released decoded-prediction JSONs. *Path convention*: all cited file paths (e.g., audit/digit_entropy.json, src/miracle/model/digit_value_head.py, scripts/analysis/...) are relative to the repository root unless explicitly noted; outputs/decoder20260511/ is the project working directory for the artifacts referenced throughout.

Appendix C Reproducibility

Code, data, per-fold splits, checkpoints (per-fold encoder and headline decoder), and every audit/*.json artifact cited in this paper are released (see the Data Availability Statement). Training uses AdamW ($\eta = 5 \times 10^{-5}$, weight decay 0.05, batch 64, up to 300 epochs, patience 75) at seed 42; the headline command accuracy is seed-robust (four-seed 5-fold means 0.8875/0.8864/0.8937/0.8949, span 0.0085, far inside the ± 0.0558 cross-fold spread; audit/seed_robustness_full_window.json). The full hyperparameter grid, per-modality retraining protocol, calibration and grammar-FSM detail, and the statistical exhibits are in the released artifacts.

Author Contributions: Conceptualization, S.S.E. and R.S.P.; methodology, S.S.E.; software, S.S.E.; validation, S.S.E.; formal analysis, S.S.E.; investigation, S.S.E.; resources, M.S.S.; data curation, S.S.E. and R.S.P.; writing—original draft, S.S.E.; writing—review and editing, R.S.P. and M.S.S.; visualization, S.S.E.; supervision, R.S.P. and M.S.S.; project administration, R.S.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The sensor datasets and analysis code generated during this study are publicly available at https://github.com/stephenseacuello/gcode_fingerprinting (accessed on 28 May 2026), at the tagged release v1.0-submission corresponding to the commit hash recorded in the repository's CITATION.cff. A pre-acceptance Zenodo DOI for the same tagged release is minted via the GitHub–Zenodo integration prior to submission and is listed in the repository's top-level README alongside the SHA256 manifest of the released checkpoints (per-fold encoder weights, headline decoder weights, and the T3.1 control). Source code for the G-Code Decoder decoder architecture, training and evaluation pipelines, audit-JSON regeneration scripts, and figure-generation scripts is released under the MIT License; the 120-file aligned corpus is released under CC BY 4.0.

Acknowledgments: The authors thank the Department of Industrial and Systems Engineering at the University of Rhode Island for providing laboratory facilities and computational resources.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

Abbreviations

The following abbreviations are used in this manuscript:

ANOVA, Analysis of Variance; AR, Autoregressive; BH-FDR, Benjamini–Hochberg False Discovery Rate; CAM, Computer-Aided Manufacturing; CI, Confidence Interval; CNC, Computer Numerical Control; CV, Cross-Validation; dw, Digit-head Loss Weight; EMI, Electromagnetic Interference; FDM, Fused Deposition Modeling; FPR, False Positive Rate; FSM, Finite-State Machine; HGB, Histogram-based Gradient Boosting; ICS, Industrial Control System; IMU, Inertial Measurement Unit; LOCO, Leave-One-Class-Out; LOMO, Leave-One-Modality-Out; LSTM, Long Short-Term Memory; LOSO, Leave-One-Sensor-Out; MAE, Mean Absolute Error; MDE, Minimum Detectable Effect; MLP, Multi-Layer Perceptron; MM-DTAE-LSTM, Multi-Modal Denoising Transformer Autoencoder with Long Short-Term Memory; NIST, National Institute of Standards and Technology; PCA, Principal Component Analysis; pp, Percentage Points; RMS, Root Mean Square; SOC, Security Operations Center; SS, Scheduled Sampling; TF, Teacher-Forced; TRL, Technology Readiness Level; VMC, Vertical Machining Center.

1. National Institute of Standards and Technology. Guide to Operational Technology (OT) Security. Technical Report SP 800-82 Rev. 3, NIST, 2023.
2. Vogl, G.W. Research Needs for Cyberphysical Systems in Machining and Machine Tools: FMNet 2024 Report. Technical report, National Institute of Standards and Technology, 2024. Future of Manufacturing Network (FMNet) 2024 workshop report.
3. Sturm, L.D.; Williams, C.B.; Camelio, J.A.; White, J.; Parker, R. Cyber-physical vulnerabilities in additive manufacturing systems. International Solid Freeform Fabrication Symposium, 2014, pp. 951–963.
4. Yampolskiy, M.; King, W.E.; Gatlin, J.; Belikovetsky, S.; Brown, A.; Skjellum, A.; Elovici, Y. Security of Additive Manufacturing: Attack Taxonomy and Survey. *Additive Manufacturing* **2018**, *21*, 431–457.
5. Zhong, R.Y.; Xu, X.; Klotz, E.; Newman, S.T. Intelligent Manufacturing in the Context of Industry 4.0: A Review. *Engineering* **2017**, *3*, 616–630. doi:10.1016/J.ENG.2017.05.015.
6. Kimmell, J.C.; Orlyanchik, V.; Sturm, L.; Dawson, J.; Taylor, C. Detecting Control Injection Attacks Using Energy Data Anomalies in Computer Numerical Control Machining. *Critical Infrastructure Protection XVIII* **2025**, *725*, 43–63.
7. Teti, R.; Jemielniak, K.; O'Donnell, G.; Dornfeld, D. Advanced monitoring of machining operations. *CIRP Annals* **2010**, *59*, 717–739.
8. Serin, G.; Sener, B.; Ozbayoglu, A.M.; Unver, H.O. Review of tool condition monitoring in machining and opportunities for deep learning. *The International Journal of Advanced Manufacturing Technology* **2020**, *109*, 953–974.

9. Al Faruque, M.A.; Chhetri, S.R.; Wan, J.; Canedo, A. Acoustic Side-Channel Attacks on Additive Manufacturing Systems. *Proceedings of the 7th ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, 2016, pp. 1–10. doi:10.1109/ICCPS.2016.7479068.
10. Song, C.; Lin, F.; Ba, Z.; Ren, K.; Zhou, C.; Xu, W. My Smartphone Knows What You Print: Exploring Smartphone-Based Side-Channel Attacks Against 3D Printers. *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 895–907. doi:10.1145/2976749.2978300.
11. Chhetri, S.R.; Canedo, A.; Al Faruque, M.A. KCAD: Kinetic Cyber-Attack Detection Method for Cyber-Physical Additive Manufacturing Systems. *35th IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, pp. 1–8. doi:10.1145/2966986.2967050.
12. Chhetri, S.R.; Canedo, A.; Al Faruque, M.A. Confidentiality Breach Through Acoustic Side-Channel in Cyber-Physical Additive Manufacturing Systems. *ACM Transactions on Cyber-Physical Systems* **2017**, 2, 3:1–3:25. doi:10.1145/3078622.
13. Jamarani, A.; Tu, Y.; Hei, X. Practitioner Paper: Decoding Intellectual Property: Acoustic and Magnetic Side-Channel Attack on a 3D Printer. *Security and Privacy in Cyber-Physical Systems and Smart Vehicles (SmartSP 2024)*. Springer, 2024, Vol. 622, *LNICST*, pp. 33–52. doi:10.1007/978-3-031-93354-7_3.
14. Chattopadhyay, T.; Ceschin, F.; Garza, M.E.; Zyunkin, D.; Chhotaray, A.; Stebner, A.P.; Zonouz, S.; Beyah, R. One Video to Steal Them All: 3D-Printing IP Theft through Optical Side-Channels. *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025. doi:10.1145/3719027.3744837.
15. Wang, J.; Ma, Y.; Zhang, L.; Gao, R.X.; Wu, D. Deep learning for smart manufacturing: Methods and applications. *Journal of Manufacturing Systems* **2018**, 48, 144–156. doi:10.1016/j.jmsy.2018.01.003.
16. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. Multimodal Sensor-Fusion Deep Learning for CNC Toolpath and Condition Classification: A Cross-Validated Ablation Study. *Machines* **2026**. Under review; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
17. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. Formalizing G-Code for Language Models: Grammar-Aware Hierarchical Tokenization of CNC Milling Toolpaths, 2026. In preparation; preprint to be deposited on the URI institutional repository and arXiv before MDPI submission, DOI/URL to be filled in at proof stage.
18. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to Sequence Learning with Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, Vol. 27.
19. Bahdanau, D.; Cho, K.; Bengio, Y. Neural Machine Translation by Jointly Learning to Align and Translate. *3rd International Conference on Learning Representations (ICLR)*, 2015.
20. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, Vol. 30.
21. Graves, A.; Mohamed, A.r.; Hinton, G. Speech Recognition with Deep Recurrent Neural Networks. *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 6645–6649.
22. Meyes, R.; Lu, M.; de Puiseau, C.W.; Meisen, T. Ablation Studies in Artificial Neural Networks. *arXiv preprint arXiv:1901.08644* **2019**.
23. Jia, F.; Lei, Y.; Lin, J.; Zhou, X.; Lu, N. Deep neural networks: A promising tool for fault characteristic mining and intelligent diagnosis of rotating machinery with massive data. *Mechanical Systems and Signal Processing* **2016**, 72–73, 303–315. doi:10.1016/j.ymssp.2015.10.025.
24. Scholak, T.; Schucher, N.; Bahdanau, D. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *Proceedings of EMNLP*, 2021.
25. Willard, B.T.; Louf, R. Efficient Guided Generation for Large Language Models. *arXiv preprint arXiv:2307.09702* **2023**.
26. Dong, Y.; Ruan, C.F.; Cai, Y.; Lai, R.; Xu, Z.; Zhao, Y.; Chen, T. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
27. Eacuello, S.S.; Prasad, R.S.; Sodhi, M.S. Source-Entropy-Correlated Recoverability of CNC Toolpath Coordinates under Low-Rate Sensing: Quantization Structure, an Operation-Class Ordering, and the 4 Hz Sub-Nyquist Floor. *Entropy* **2026**. Companion paper.

- 1291 28. Zhu, K.; Wong, Y.S.; Hong, G.S. Wavelet analysis of sensor signals for tool condition monitoring: A review
1292 and some new results. *International Journal of Machine Tools and Manufacture* **2009**, *49*, 537–553.
- 1293 29. Mohanraj, T.; Shankar, S.; Rajasekar, R.; Sakthivel, N.; Pramanik, A. Tool condition monitoring techniques
1294 in milling process — a review. *Journal of Materials Research and Technology* **2020**, *9*, 1032–1042.
- 1295 30. Zhao, R.; Yan, R.; Chen, Z.; Mao, K.; Wang, P.; Gao, R.X. Deep learning and its applications
1296 to machine health monitoring. *Mechanical Systems and Signal Processing* **2019**, *115*, 213–237.
1297 doi:10.1016/j.ymssp.2018.05.050.
- 1298 31. Janssens, O.; Slavkovikj, V.; Vervisch, B.; Stockman, K.; Loccufier, M.; Verstockt, S.; Van de Walle, R.;
1299 Van Hoecke, S. Convolutional Neural Network Based Fault Detection for Rotating Machinery. *Journal of*
1300 *Sound and Vibration* **2016**, *377*, 331–345. doi:10.1016/j.jsv.2016.05.027.
- 1301 32. Bhandari, B. Comparative Study of Popular Deep Learning Models for Machining Roughness Classification
1302 Using Sound and Force Signals. *Micromachines* **2021**, *12*, 1484. doi:10.3390/mi12121484.
- 1303 33. Stathatos, E.; Tzimas, E.; Benardos, P.; Vosniakos, G.C. Convolutional Neural Networks for Raw Signal
1304 Classification in CNC Turning Process Monitoring. *Sensors* **2024**, *24*, 1390.
- 1305 34. Coelho, P.J.; Athar, A.; Mozumder, M.A.I.; Ali, S.; Kim, H.C. Deep Learning-Based Anomaly
1306 Detection Using One-Dimensional Convolutional Neural Networks (1D CNN) in Machine Centers
1307 (MCT) and Computer Numerical Control (CNC) Machines. *PeerJ Computer Science* **2024**, *10*, e2389.
1308 doi:10.7717/peerj-cs.2389.
- 1309 35. Lahat, D.; Adali, T.; Jutten, C. Multimodal Data Fusion: An Overview of Methods, Challenges, and
1310 Prospects. *Proceedings of the IEEE* **2015**, *103*, 1449–1477. doi:10.1109/JPROC.2015.2460697.
- 1311 36. Ramachandram, D.; Taylor, G.W. Deep Multimodal Learning: A Survey on Recent Advances and Trends.
1312 *IEEE Signal Processing Magazine* **2017**, *34*, 96–108. doi:10.1109/MSP.2017.2738401.
- 1313 37. Cao, L.; Li, J.; Zhang, L.; Luo, S.; Li, M.; Huang, X. Cross-attention-based multi-sensing signals fusion
1314 for penetration state monitoring during laser welding of aluminum alloy. *Knowledge-Based Systems* **2023**,
1315 *261*, 110212. doi:10.1016/j.knosys.2022.110212.
- 1316 38. Wang, H.; Wang, S.; Sun, W.; Xiang, J. Multi-Sensor Signal Fusion for Tool Wear Condition Monitoring
1317 Using Denoising Transformer Auto-Encoder ResNet. *Journal of Manufacturing Processes* **2024**, *124*, 1227–1238.
- 1318 39. Bergs, T.; Gierlings, S.; Auerbach, T.; Klink, A.; Schraknepper, D.; Augspurger, T. The Concept of Digital
1319 Twin and Digital Shadow in Manufacturing. *Procedia CIRP* **2021**, *101*, 81–84.
- 1320 40. Tsanousa, A.; Bektsis, E.; Kyriakopoulos, C.; Gómez González, A.; Leturiondo, U.; Gialampoukidis, I.;
1321 Karakostas, A.; Vrochidis, S.; Kompatsiaris, I. A Review of Multisensor Data Fusion Solutions in Smart
1322 Manufacturing: Systems and Trends. *Sensors* **2022**, *22*, 1734. doi:10.3390/s22051734.
- 1323 41. Wen, Q.; Zhou, T.; Zhang, C.; Chen, W.; Ma, Z.; Yan, J.; Sun, L. Transformers in Time Series: A Survey.
1324 Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI), 2023, pp.
1325 6778–6786. doi:10.24963/ijcai.2023/759.
- 1326 42. Liang, J.; Huang, W.; Xia, F.; Xu, P.; Hausman, K.; Ichter, B.; Florence, P.; Zeng, A. Code as Policies:
1327 Language Model Programs for Embodied Control. IEEE International Conference on Robotics and
1328 Automation (ICRA), 2023, pp. 9493–9500.
- 1329 43. Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Computation* **1997**, *9*, 1735–1780.
1330 doi:10.1162/neco.1997.9.8.1735.
- 1331 44. Malhotra, P.; Vig, L.; Shroff, G.; Agarwal, P. Long Short Term Memory Networks for Anomaly Detection in
1332 Time Series. 23rd European Symposium on Artificial Neural Networks (ESANN), 2015, pp. 89–94.
- 1333 45. Vincent, P.; Larochelle, H.; Lajoie, I.; Bengio, Y.; Manzagol, P.A. Stacked Denoising Autoencoders: Learning
1334 Useful Representations in a Deep Network with a Local Denoising Criterion. *Journal of Machine Learning*
1335 *Research* **2010**, *11*, 3371–3408.
- 1336 46. Poesia, G.; Polozov, O.; Le, V.; Tiwari, A.; Soares, G.; Meek, C.; Gulwani, S. Synchronesh: Reliable Code
1337 Generation from Pre-trained Language Models. International Conference on Learning Representations
1338 (ICLR), 2022.
- 1339 47. Beurer-Kellner, L.; Fischer, M.; Vechev, M. Prompting Is Programming: A Query Language for Large
1340 Language Models. *Proceedings of the ACM on Programming Languages (PACMPL)*, *OOPSLA* **2023**,
1341 *7*, 1946–1969.

48. Geng, S.; Josifoski, M.; Peyrard, M.; West, R. Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 10932–10952.
49. Jignasu, A.; Marshall, K.; Ganapathysubramanian, B.; Balu, A.; Hegde, C.; Krishnamurthy, A. Towards Foundational AI Models for Additive Manufacturing: Language Models for G-Code Debugging, Manipulation, and Comprehension. *arXiv preprint arXiv:2309.02465* **2023**.
50. Abdelaal, M.; Lokadjaja, S.; Engert, G. GLLM: Self-Corrective G-Code Generation using Large Language Models with User Feedback. *Proceedings of the 21st Conference on Database Systems for Business, Technology and Web (BTW), Industrial Track*, 2025.
51. Wang, Z.; Jadhav, Y.; Pak, P.; Barati Farimani, A. Image2Gcode: Image-to-G-code Generation for Additive Manufacturing Using Diffusion-Transformer Model. *arXiv preprint arXiv:2511.20636* **2025**.
52. Jeon, J.; Sim, Y.; Lee, H.; Han, C.; Yun, D.; Kim, E.; Nagendra, S.L.; Jun, M.B.; Kim, Y.; Lee, S.W.; Lee, J. ChatCNC: Conversational machine monitoring via large language model and real-time data retrieval augmented generation. *Journal of Manufacturing Systems* **2025**, *79*, 504–514.
53. Belikovetsky, S.; Yampolskiy, M.; Toh, J.; Gatlin, J.; Elovici, Y. dr0wned – Cyber-Physical Attack with Additive Manufacturing. *11th USENIX Workshop on Offensive Technologies (WOOT)*, 2017.
54. Zonouz, S.; Rrushi, J.; McLaughlin, S. Detecting Industrial Control Malware Using Automated PLC Code Analytics. *IEEE Security & Privacy* **2014**, *12*, 40–47. doi:10.1109/MSP.2014.113.
55. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; others. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **2011**, *12*, 2825–2830.
56. Ba, J.L.; Kiros, J.R.; Hinton, G.E. Layer Normalization. *arXiv preprint arXiv:1607.06450* **2016**.
57. Srivastava, N.; Hinton, G.; Krizhevsky, A.; Sutskever, I.; Salakhutdinov, R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* **2014**, *15*, 1929–1958.
58. Bengio, S.; Vinyals, O.; Jaitly, N.; Shazeer, N. Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, Vol. 28.
59. Loshchilov, I.; Hutter, F. Decoupled Weight Decay Regularization. *7th International Conference on Learning Representations (ICLR)*, 2019.
60. Szegedy, C.; Vanhoucke, V.; Ioffe, S.; Shlens, J.; Wojna, Z. Rethinking the Inception Architecture for Computer Vision. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2818–2826. doi:10.1109/CVPR.2016.308.
61. Lin, T.Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. Focal Loss for Dense Object Detection. *IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
62. Virtanen, P.; Gommers, R.; Oliphant, T.E.; Haberland, M.; Reddy, T.; Cournapeau, D.; Burovski, E.; Peterson, P.; Weckesser, W.; Bright, J.; others. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods* **2020**, *17*, 261–272. doi:10.1038/s41592-019-0686-2.
63. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794. doi:10.1145/2939672.2939785.
64. He, T.; Zhang, J.; Zhou, Z.; Glass, J. Quantifying Exposure Bias for Neural Language Generation. *arXiv preprint arXiv:1905.10617* **2019**.
65. Ott, M.; Auli, M.; Grangier, D.; Ranzato, M. Analyzing Uncertainty in Neural Machine Translation. *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 3956–3965.
66. Ranzato, M.; Chopra, S.; Auli, M.; Zaremba, W. Sequence level training with recurrent neural networks. *International Conference on Learning Representations (ICLR)*, 2016.
67. Holtzman, A.; Buys, J.; Du, L.; Forbes, M.; Choi, Y. The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*, 2020.
68. Welleck, S.; Kulikov, I.; Roller, S.; Dinan, E.; Cho, K.; Weston, J. Neural text generation with unlikelihood training. *International Conference on Learning Representations (ICLR)*, 2020.